

Part 5 · continued

5.3 · Underneath signup — what the system does

Repo root: `/Users/matan/Projects/soccer`. All paths below are absolute. **Live hosting domain:** `https://teamderfc.web.app` — there is no custom domain. Nothing in this surface depends on it, but the email-verification and password-reset emails Firebase sends are Firebase-hosted (`/Users/matan/Projects/soccer/src/firebase/auth.ts:171`, `:202`), so their links carry a `firebaseapp.com` action handler, not a Teamder domain. **Report scope:** the invisible half of signup. Everything a consultant cannot see by tapping through the app.

1. Every auth provider offered

There are **five** distinct entry paths into an identity, exposed by four buttons on one screen.

1.1 THE BUTTONS ON `SIGNINSCREEN`

`/Users/matan/Projects/soccer/src/screens/auth/SignInScreen.tsx:173-275`

Order on screen	Label (verbatim)	Gloss	Handler	Platform
1	<code>'המשך עם Google'</code>	"Continue with Google"	<code>handlePress</code> — <code>SignInScreen.tsx:44-93</code>	Android + iOS
2	<code>'המשך עם Apple'</code>	"Continue with Apple"	<code>handleApple</code> — <code>SignInScreen.tsx:97-139</code>	iOS only — gated by <code>Platform.OS === 'ios'</code> at <code>SignInScreen.tsx:212</code>
3	<code>'המשך עם מייל'</code>	"Continue with email"	navigates to <code>EmailAuth</code> — <code>SignInScreen.tsx:236-252</code>	both
4	<code>'המשך כאורח'</code>	"Continue as guest"	<code>handleGuest</code> — <code>SignInScreen.tsx:143-155</code>	both

Strings: `/Users/matan/Projects/soccer/src/i18n/he.ts:1795` `signInGoogle: 'המשך עם Google', :1796` `signInApple: 'המשך עם Apple', :1797` `signInEmail: 'המשך עם מייל', :1798` `signInGuest: 'המשך כאורח'.`

Screen copy: `he.ts:1793` `signInTitle: 'בואו נכיר' ["let's get started"]`, `he.ts:1794` `signInSubtitle: 'התחבר כדי להירשם, להצטרף למועדון ולעקוב אחרי הסטטיסטיקות שלך' , he.ts:1799` `signInPrivacy: 'באמצעות התחברות אתה מסכים לתנאי השימוש' ["by signing in you agree to the terms of use"]` — rendered as plain `<Text>` at `SignInScreen.tsx:272`, **not a link**; nothing in this file opens a terms document.

Note the **title/subtitle grammar mix**: the title is plural/polite (`בואו`) and the subtitle is masculine-singular imperative (`התחבר`). Both are on screen at the same time.

Note also `he.ts:1846` `profileTitle: 'בוא נכיר'` and `he.ts:2839` `psProfileTitle: 'בוא נכיר'` — the *next* screen's title ("let's get acquainted") is one character away from the sign-in title `'בואו נכיר'`.

1.2 THE FIVE SERVICE-LAYER SIGNUP PATHS

All in `/Users/matan/Projects/soccer/src/services/userService.ts`:

1. `signInAsGuest` — `userService.ts:61-67` → `fbSignInAnonymously` (`/Users/matan/Projects/soccer/src/firebase/auth.ts:407-412`). **Writes no `/users` doc at all.**
2. `signInWithGoogle` — `userService.ts:167-212` → `signInWithGoogle` (`auth.ts:79-128`), native picker via `@react-native-google-signin/google-signin`.
3. `signInWithApple` — `userService.ts:214-256` → `signInWithApple` (`auth.ts:331-399`), `expo-apple-authentication` with a SHA-256 hashed nonce (`auth.ts:343-349`).
4. `signInWithEmail` (existing account) — `userService.ts:263-298` → `auth.ts:148-159`.
5. `signUpWithEmail` (new account) — `userService.ts:306-343` → `auth.ts:161-197`.

Plus a sixth, non-button path: `getCurrentUser lazy-create` — `userService.ts:140-158`. On cold start, if Firebase Auth restores a user but `/users/{uid}` does not exist, the doc is created there. Comment at `userService.ts:134-139` states the two cases it covers: a brand-new sign-in whose `setDoc` never ran, and recovery after a previous launch left an Auth user with no doc.

1.3 GUEST — WHAT AN ANONYMOUS SESSION ACTUALLY IS

`buildGuestUser` — `userService.ts:44-53`:

```
{ id: uid, name: '', avatarId: pickRandomAvatarId(), createdAt: Date.now(),
  onboardingCompleted: true, isGuest: true }
```

This object exists **only in memory**. It is never written to Firestore, and `cacheAuthUser` explicitly refuses to persist it (`userService.ts:881-883`). `onboardingCompleted: true` is a lie told to the navigator so it skips the profile gate (`userService.ts:90-91`, `/Users/matan/Projects/soccer/src/navigation/RootNavigator.tsx:267`).

A guest hitting any account action gets a dialog from `ensureNotGuest`

(`/Users/matan/Projects/soccer/src/utils/guestGate.ts:39-64`): - Title `he.ts:1801` `guestRegisterTitle`: 'נדרשת הרשמה' ["registration required"] - Default body `he.ts:1802` `guestRegisterBody`: 'כדי להשתמש בתכונה כדי ליצור חשבון. רוצה להירשם עכשיו?' - Per-action bodies: `he.ts:1803` `guestRegisterJoinGame`: 'כדי להצטרף'; `:1804` `guestRegisterJoinCommunity`: 'כדי לשלוח הודעה למנהל'; `:1805` `guestRegisterChatAdmin`: 'כדי ליצור צ'אט חשבון. רוצה להירשם עכשיו?'; `:1806` `guestRegisterCreate`: 'כדי ליצור צ'אט חשבון. רוצה להירשם עכשיו?'; `:1807` `guestRegisterChat`: 'כדי להשתמש בצ'אט חשבון. רוצה להירשם עכשיו?'; - Buttons: `he.ts:52` `cancel`: 'בטל' and `he.ts:1808` `guestRegisterCta`: 'הרשמה'

Choosing 'הרשמה' **signs the guest out entirely** (`guestGate.ts:58`) — the whole navigator swaps and they land back on `SignInScreen`. The action they were attempting is stashed first as a pending invite (`guestGate.ts:51-57`) so `RootNavigator`'s consumer can resume it after signup (`RootNavigator.tsx:63-166`).

2. The exact document written to `/users/{uid}` on first signup

This is the single most commonly mis-stated fact about this codebase: **the object built in `userService` is not the object stored**. Every write goes through `docs.user(uid)`

(`/Users/matan/Projects/soccer/src/firebase/firestore.ts:2039-2041`), which is `doc(col.users(),`

uid), and `col.users()` carries `.withConverter(userConverter)` (`firestore.ts:1911–1913`). So `setDoc(ref, fresh)` writes `userConverter.toFirestore(fresh)` (`firestore.ts:183–264`).

2.1 THE OBJECT THE SERVICE BUILDS

Google — `userService.ts:181–188` ; Apple — `:228–239` ; email sign-up — `:320–327` ; email sign-in lazy-create — `:277–284` ; cold-start lazy-create — `:140–147` .

Field	Google :183–187	Apple :234–238	Email sign-up :322–326	Email sign-in lazy :279–283	Cold-start lazy :142–146
name	<code>fbUser.displayName ?? ''</code>	<code>fbUser.displayName ?? fullName ?? ''</code>	<code>''</code> hardcoded	<code>fbUser.displayName ?? ''</code>	<code>fbUser.displayName ?? ''</code>
email	<code>fbUser.email ?? undefined</code>	<code>fbUser.email ?? undefined</code>	<code>fbUser.email ?? email.trim()</code>	<code>fbUser.email ?? email.trim()</code>	<code>fbUser.email ?? undefined</code>
avatarId	<code>pickRandomAvatarId()</code>	same	same	same	same
createdAt	<code>Date.now()</code>	same	same	same	same
onboardingCompleted	<code>false</code>	<code>false</code>	<code>false</code>	<code>false</code>	<code>false</code>

Apple's `fullName` is only available on the very first authorization; the comment at `userService.ts:230–233` and `auth.ts:326–329` both say so. `auth.ts:378–383` joins `givenName` + `familyName` .

`pickRandomAvatarId()` — `/Users/matan/Projects/soccer/src/data/avatars.ts:70–72` — picks uniformly from the 24 entries in `AVATARS` (`avatars.ts:23–55`), each an emoji glyph on a coloured circle (`a01` 🏠 ... `a24` 🍌). **Every new account therefore arrives with a randomly-assigned emoji avatar before the user has chosen anything.**

2.2 THE DOCUMENT THAT ACTUALLY LANDS IN FIRESTORE

Produced by `userConverter.toFirestore` (`firestore.ts:183–264`). For a brand-new Google account:

Firestore field	Initial value	Source line
name	the provider display name, or <code>''</code>	<code>firestore.ts:185</code>
email	the address, or <code>null</code>	<code>firestore.ts:186</code>
avatarId	<code>'a01'</code> ... <code>'a24'</code> (random)	<code>firestore.ts:187</code>
position	<code>null</code>	<code>firestore.ts:188</code>
createdAt	device-clock epoch ms	<code>firestore.ts:189</code>
updatedAt	<code>Date.now()</code> (since <code>u.updatedAt</code> is undefined)	<code>firestore.ts:190</code>
onboardingCompleted	<code>false</code>	<code>firestore.ts:191</code>
availability	<code>null</code>	<code>firestore.ts:192–217</code>
stats	<code>null</code>	<code>firestore.ts:218–243</code>

Firestore field	Initial value	Source line
notificationPrefs	null	firestore.ts:250
dmFriendsOnly	false	firestore.ts:251
newGameSubscriptions	[]	firestore.ts:252
personalGroupId	null	firestore.ts:253
jersey	null	firestore.ts:254
achievements	null	firestore.ts:255
discipline	null	firestore.ts:256
invitedBy	null	firestore.ts:257
invitedByType	null	firestore.ts:258
invitedByTargetId	null	firestore.ts:259

Fields deliberately NOT written on create, each for a stated reason: - `photoUrl` — absent from `toFirestore` entirely. The comment at `firestore.ts:274` says "photoUrl kept readable for legacy docs; never written by new code." **Consequence: the Google account photo is discarded at signup.** A new user's picture is the random emoji until they upload one on the onboarding screen. - `fcmTokens` — moved to the self-only subdoc `/users/{uid}/private/push`; `firestore.ts:244–249` explains that writing them on the world-readable doc re-leaked every device token to any signed-in user. - `invitedAt` — written separately with `serverTimestamp()` so a re-save cannot clobber the server time (`firestore.ts:260–263`, written at `userService.ts:934`). - `friends` — read but never written; owned by the server-side accept/remove callables (`firestore.ts:294–298`). - `qa`, `platform`, `lastSeenAt`, `acquisition`, `lastBroadcastAt` — none exist at create; each is added later by a different writer.

2.3 NOTIFICATION PREFERENCES ON DAY ONE

`notificationPrefs` is `null` on the document. The **effective** defaults come from `defaultNotificationPrefs` (`/Users/matan/Projects/soccer/src/types/index.ts:405–424`): all 18 types default `true` except `growthMilestone: false` (`types/index.ts:414`). `marketingPush: true` (`types/index.ts:423`) — a new user is opted **in** to marketing push by default, with nothing on the signup screens saying so.

2.4 WHAT FIRESTORE RULES REQUIRE OF THAT CREATE

`/Users/matan/Projects/soccer/firestore.rules:262–277`:

```
allow create: if isSelf(uid) &&
  (!('invitedBy' in request.resource.data) || request.resource.data.invitedBy != uid) &&
  isOptionalShortString(request.resource.data, 'name', 60) &&
  nameNotReserved(request.resource.data) &&
  nameNotEmail(request.resource.data) &&
  isOptionalShortString(request.resource.data, 'email', 200) &&
  isOptionalShortString(request.resource.data, 'photoUrl', 2048) &&
  isOptionalShortString(request.resource.data, 'avatarId', 80);
```

`isSelf` — `firestore.rules:31-33`. `isOptionalShortString` — `firestore.rules:124-127` (missing **or null** or short enough), which is what lets the converter's `null` placeholders through. `allow read: if isSignedIn()` (`firestore.rules:260`) — **every signed-in user can read every user document**, including a brand-new one. `allow delete: if false` (`firestore.rules:343`).

Two size mismatches worth naming: the rule caps `name` at **60** characters (`firestore.rules:272`) while both name inputs cap at **40** (`PostSignInOnboardingScreen.tsx:169`, `ProfileSetupScreen.tsx:81`).

3. Validation on the name field

There are **four** layers, and they do not all say the same thing.

3.1 LAYER 1 — THE INPUT ITSELF

- `PostSignInOnboardingScreen.tsx:164-172`: `maxLength={40}`, `required`, label `he.profileName`, placeholder `he.profileNamePlaceholder`. **No** `returnKeyType` / **submit wiring**.
- `ProfileSetupScreen.tsx:76-88`: `maxLength={40}`, `required`, plus `returnKeyType="done"` and `onSubmitEditing` that saves (`ProfileSetupScreen.tsx:84-87`).
- Save CTA enabled only when `name.trim().length > 0` — `PostSignInOnboardingScreen.tsx:61`, `ProfileSetupScreen.tsx:27`.

Strings: `he.ts:1847` `profileName: 'שם' ["name"]`, `he.ts:1848` `profileNamePlaceholder: 'איך לקרוא לך?'` ["what should we call you?"], `he.ts:1849` `profileNameRequired: 'שם הוא שדה חובה'` ["name is a required field"] — `profileNameRequired` **has no call site in either signup screen**; the CTA simply stays disabled instead.

3.2 LAYER 2 — SANITISATION

`sanitizeDisplayString` — `/Users/matan/Projects/soccer/src/utils/validate.ts:111-119`. Strips C0 control characters + DEL, then zero-width and bidirectional-override characters (U+200B–U+200F, U+202A–U+202E, U+2060–U+206F, U+FEFF), then trims. Called at `userService.ts:363` (`completeOnboarding`) and `userService.ts:730` (`updateProfile`). The comment at `userService.ts:360-362` notes onboarding used to only `trim()`, so impersonation and layout-reversal characters persisted.

3.3 LAYER 3 — THE RESERVED-BRAND AND EMAIL-SHAPED CHECKS (CLIENT)

`/Users/matan/Projects/soccer/src/utils/officialAccount.ts`.

Reserved brand — `officialAccount.ts:32`:

```
export const RESERVED_NAME_RE = /teamder|טימדר|טיאמדר|טימדר/i;
```

`isReservedName` (`officialAccount.ts:47-52`) strips whitespace, dots, underscores and hyphens first (`.replace(/[.\s._-]/g, '')`) so `"T e a m d e r"` and `"team-der"` fall to the same test. Deliberately a **substring** match (`officialAccount.ts:25-32`): `"Teamder Support"`, `"teamder.app"` and `"צוות טימדר"` are all blocked. The Hebrew variants are `טימדר`, `טיאמדר`, `טימדר`.

Email-shaped name — `officialAccount.ts:62`:

```
const EMAIL_NAME_RE = /^[@\s]@[^@\s]+\.[a-z]{2,}/i;
```

`isEmailLikeName` — `officialAccount.ts:87–89`. The doc comment (`officialAccount.ts:64–86`) records why it exists: Google Play's **pre-launch report** robot (Firebase Test Lab) is handed the demo credentials from Play Console's "App access" page and types them into every text input it meets, including this name field. **47 accounts named** `appstore.review@teamder.app` **accumulated between 22.06 and 19.09.2026**, all Android, clustering on release days. Two of them played in a finished game of a real seven-player club and sit inside its statistics; a third was in that club's pending queue 75 seconds after signing up. Of 676 live accounts on 19.09.2026, not one real person had chosen an email-shaped name.

Thrown as opaque codes: `userService.ts:368` `throw new Error('RESERVED_NAME')`, `userService.ts:373` `throw new Error('EMAIL_NAME')` (in `completeOnboarding`); mirrored at `userService.ts:731–732` (in `updateProfile`).

3.4 LAYER 4 — FIRESTORE RULES (THE ACTUAL ENFORCEMENT)

`firestore.rules:148–152` `nameNotReserved`:

```
!(data.name.lower().replace(['\\s._-'], '').matches('.*(teamder|תימדר|טיאמדר|טימדר).*'))
```

`firestore.rules:182–185` `nameNotEmail`:

```
!(data.name.lower().matches('.*[^\s]@[^\s]+[.][a-z]{2,}.**'))
```

On **update** both are relaxed to fire only when the name actually changes — `nameNotNewlyReserved` (`firestore.rules:203–207`) and `nameNotNewlyEmail` (`firestore.rules:191–195`) — so an account already carrying such a name is not locked out of unrelated writes (FCM token refresh, availability). Rationale at `firestore.rules:187–190` and `:196–202`.

The rules comment at `firestore.rules:154–181` carries the same 47-account history and adds the specific hole that was closed: `nameNotReserved` **used to exempt any name containing '@'**, and `appstore.review@teamder.app` contains "teamder", so the `@` escape hatch was the entire vulnerability.

Pinned cases live in `/Users/matan/Projects/soccer/tests/rules/displayName.test.mjs` — blocked at `:95–100` (`appstore.review@teamder.app`, `hazelblake.54551@gmail.com`, uppercase, `+tag`, padded, and an address **embedded** in a Hebrew name `'someone@gmail.com מתן'`); allowed at `:110–113` (`'עידן @ נחלים'`, `'DJ @Khaled'`, `user@localhost`, `a@b.c`).

3.5 THE HEBREW ERROR STRINGS — AND WHERE THEY ARE AND ARE NOT SHOWN

Code	String (verbatim)	he.ts line
RESERVED_NAME	<code>officialNameTaken: 'השם הזה שמור לחשבון הרשמי של Teamder. בחרו שם אחר.'</code>	<code>he.ts:640–641</code>
EMAIL_NAME	<code>emailNameNotAllowed: 'כתובת אימייל היא לא שם. איך קוראים לכם.'</code>	<code>he.ts:642–643</code>
generic	<code>profileSaveError: 'לא הצלחנו לשמור את הפרטים. בדוק את החיבור ונסה שוב.'</code>	<code>he.ts:189</code>
dialog title	<code>error: 'שגיאה'</code>	<code>he.ts:188</code>

`ProfileSetupScreen` **maps all three correctly** — `ProfileSetupScreen.tsx:44–51`.

`PostSignInOnboardingScreen` **does not**. Its catch block is `PostSignInOnboardingScreen.tsx:121–130`, and it shows `appAlert(he.error, he.signInFailed)` unconditionally (`:127`). `he.signInFailed` is `he.ts:1264` `'ההתחברות נכשלה. נסה שוב.'` `["sign-in failed. try again."]`. So on the screen **every new user actually passes through**, a user who types "Teamder" or their own email address as their name is told that *sign-in* failed — a sentence about a step they already completed, with no indication that the name is the problem, and the CTA re-enables so they can tap it again forever. `he.profileSaveFailed` (`he.ts:2817` `'שמירת הפרופיל נכשלה. נסה שוב.'`) exists and is not used here either.

`ProfileSetupScreen` — the screen with the correct messages — is reachable only when `onboardingCompleted === true` **and** `name` is empty (`RootNavigator.tsx:267–271`), i.e. legacy accounts. Since `completeOnboarding` sets both in one write (`userService.ts:392–396`), a new user in 2026 never sees it.

4. What fires automatically on signup

Grouped by when it runs relative to the user seeing the next screen.

4.1 INSIDE THE SIGNUP CALL, AWAITED — BLOCKS THE TRANSITION

For Google (`userService.ts:167–212`), Apple (`:214–256`) and email **sign-up** (`:306–343`), in this exact order:

1. `getDoc(ref)` — one read to check whether the doc already exists (`userService.ts:175`, `:222`, `:314`).
2. `setDoc(ref, fresh)` — the create (`:194`, `:241`, `:329`).
3. `applyInviteAttributionIfFresh(fresh.id)` — `userService.ts:208`, `:252`, `:339`; implementation `userService.ts:908–940`. Reads the stashed pending invite, re-reads the user doc, and if an inviter exists writes: `{ invitedBy, invitedByType, invitedByTargetId, invitedAt: serverTimestamp() }` (`userService.ts:930–935`). Hard bails: mock mode, no pending invite, no `invitedBy`, self-invite, or `invitedBy` already set (`userService.ts:912–920`). A generic app invite has no target, so `'app'` is substituted so the rules "type+target present" guard passes (`userService.ts:922–925`). `serverTimestamp()` not `Date.now()` so a wrong device clock cannot corrupt the ordering (`userService.ts:926–929`).
4. `applyAcquisitionIfFresh(fresh.id)` — `userService.ts:209`, `:253`, `:340`; implementation `:949–978`. Set-once. Writes: `acquisition: { source, campaign?, linkId?, gameId?, at: Date.now() }` (`userService.ts:965–973`). `source` is the channel label (whatsapp/facebook/...), `linkId` the specific tracked link `al_...` — shapes at `/Users/matan/Projects/soccer/src/services/storage.ts:67–73`.
5. `cacheAuthUser(fresh)` — `userService.ts:210`, `:254`, `:341`; implementation `:879–888`. Writes the user JSON to AsyncStorage key `'footy.auth.user'` (`storage.ts:8`) as the offline cold-start fallback.

Both attribution helpers are awaited, so on a slow network the user stares at the sign-in button's spinner through two extra Firestore reads and up to two writes. Both swallow their own errors (`userService.ts:936–939`, `:974–977`) so they cannot fail the signup — but they can delay it.

Asymmetry: `signInWithEmail` (`userService.ts:263–298`) **does not call either helper** — it goes straight from `setDoc` (`:286`) to `cacheAuthUser` (`:296`). So if an account is first materialised through the sign-in path rather than the sign-up path, its referral and acquisition attribution is silently lost. The cold-start lazy-create *does* call both (`userService.ts:155–156`).

4.2 FIRED IN PARALLEL, NOT AWAITED

Joryio identify — `/Users/matan/Projects/soccer/src/firebase/auth.ts:466-474`. The `onAuthStateChanged` listener registered by `waitForAuthRestore` (`auth.ts:442-481`) stays alive after the first `null` emission specifically so a later sign-in still reaches Joryio. On sign-in it calls:

```
joryio.identify(user.uid, { email: user.email ?? undefined, name: user.displayName ?? undefined })
```

`identify` (`/Users/matan/Projects/soccer/src/services/joryio.ts:130-150`) also stamps `platform` and `appVersion` as attributes (`joryio.ts:146-147`). The comment at `auth.ts:450-455` records the bug this fixed: on a fresh install the first emission is `null`, `unsub()` used to run right there, and "a day-one user spent their entire first session as an anonymous record with no email and no `push_permission` — which is precisely the attribute the onboarding journey branches on."

Native auth mirror — `mirrorToNativeAuth` at `auth.ts:112` (Google) and `auth.ts:387-389` (Apple); `mirrorEmailToNativeAuth` at `auth.ts:157 / :174` (email, Android only — `auth.ts:225`). Establishes a parallel `@react-native-firebase/auth` session so the home widget and Wear relay can write the timer (`auth.ts:239-252`).

Email verification — `sendEmailVerification(cred.user)` at `auth.ts:171`, explicitly `.catch()`-ed and never awaited. Comment at `auth.ts:133-135`: verification "only proves inbox control, not that the address is 'real'", so usage is not blocked on it. **Nothing in the app ever checks** `emailVerified` for the signup flow.

4.3 ANALYTICS EVENTS

All go to **two sinks at once** — Firebase Analytics and Joryio — via `logEvent` (`/Users/matan/Projects/soccer/src/services/analyticsService.ts:569-591`): `void joryio.track(name, cleaned)` at `:583`, then `analytics().logEvent(...)` at `:585-586`. `cleanParams` stamps `platform` on every event (`analyticsService.ts:598-612`). Joryio renames some events on the way out via `NAME_OVERRIDES` (`joryio.ts:71-89`) — relevant here: `onboarding_completed` → `'Onboarding Completed'` (`joryio.ts:85`).

In firing order for a Google signup:

Event constant	Wire name	Fired at
<code>SignInAttempted</code>	<code>sign_in_attempted</code> (<code>analyticsService.ts:319</code>)	<code>SignInScreen.tsx:45</code> , params { method: 'google' }
<code>SignInSuccess</code>	<code>sign_in_success</code> (<code>analyticsService.ts:24</code>)	<code>/Users/matan/Projects/soccer/src/store/userStore.ts:133</code>
<code>PushPermissionResult</code>	<code>push_permission_result</code> (<code>analyticsService.ts:332</code>)	<code>/Users/matan/Projects/soccer/src/services/notificationsService.ts:676-679</code> , params { granted, can_ask_again }
<code>ProfileCreated</code>	<code>profile_created</code> (<code>analyticsService.ts:30</code>)	<code>userStore.ts:256</code>
<code>OnboardingCompleted</code>	<code>onboarding_completed</code> (<code>analyticsService.ts:27</code>)	<code>userStore.ts:257</code>

Guest signup logs `SignInSuccess` with { method: 'guest' } (`userStore.ts:145`) — so **guests are counted as successful sign-ins in both analytics systems**, with only a param distinguishing them.

Optional events on the onboarding screen: `PhotoUploaded` `photo_uploaded` (`PostSignInOnboardingScreen.tsx:95`), `AvatarChanged` `avatar_changed` (`:107–110`, `params { source: 'onboarding', avatarId }`), `PhotoUploadAbandoned` `photo_upload_abandoned` (`:74–77`), `PhotoUploadFailed` `photo_upload_failed` (`:79–82`), `ProfileSaveFailed` `profile_save_failed` (`:123–126`).

Email-path events: `AuthModeSwitched` `auth_mode_switched` (`EmailAuthScreen.tsx:65–68`), `SignInProviderConflict` `sign_in_provider_conflict` (`:91`), `SignInFailed` `sign_in_failed` (`:101–105`), `PasswordResetRequested` `password_reset_requested` (`:139–142`, `:148`, `:155`, `:158`, `:162`).

4.4 PUSH PERMISSION — WHEN THE OS PROMPT APPEARS

`RootNavigator.tsx:235–248`. The effect keys on `currentUser?.id`, so it fires **the instant** `currentUser` is set — i.e. immediately after `signInWithGoogle` resolves, at the same moment `RootNavigator` renders `PostSignInOnboardingScreen`. So on a real device the OS push-permission dialog lands **on top of the "בוא נכיר" profile screen**, before the user has typed their name.

Guests are explicitly skipped (`RootNavigator.tsx:240`) with the stated reason at `:237–239`: an anonymous session should not trigger the prompt (App Store 5.1.1 friction) and would only register an orphan token.

`requestAndRegisterPushToken` (`notificationsService.ts:634–696`): checks existing permission (`:671`), requests if `canAskAgain` (`:673–680`), fetches the FCM token (`:685`), then `registerDeviceToken` (`:687`). `registerDeviceToken` (`notificationsService.ts:475–510`) does three things: mirrors the token to Joryio (`:479`), writes `{ fcmTokens: arrayUnion(token), devices: {...}, updatedAt }` to `/users/{uid}/private/push` (`:481–497`), and prunes any 64-hex APNs tokens (`:509`). `joryio.registerPushToken` (`joryio.ts:239–259`) also sets the `push_permission` attribute (`:252–257`).

There is **no pre-permission priming screen anywhere in this flow** — no explanation of why notifications matter before the OS dialog.

4.5 PRESENCE PING

`touchPresence` — `userService.ts:987–1005`, called fire-and-forget from `getCurrentUser` at `:126` when the doc already exists. Writes `{ platform: Platform.OS, lastSeenAt: now }`. Throttled to once per 6h (`userService.ts:987`, `:992–995`) and gated behind the `feature_campaigns` remote-config kill-switch (`:991`). **It does not run on the signup path** — only on subsequent launches — so `platform` and `lastSeenAt` are absent from a day-zero user document.

4.6 CLOUD FUNCTIONS THAT FIRE ON THE NEW /USERS DOC

Exactly **one** Firestore trigger fires on user creation. `grep` over `/Users/matan/Projects/soccer/functions/src` for `onDocumentCreated` returns one match on the users collection:

`onNewUserJoined` — `/Users/matan/Projects/soccer/functions/src/index.ts:12829–12896`.

- Reads `name`; falls back to `'משתמש חדש'` ["new user"] when the name is empty or the tombstone `משתמש שהוסר` (`index.ts:12833–12834`).
- **Polls for attribution:** because `applyInviteAttributionIfFresh` / `applyAcquisitionIfFresh` land as *separate client updates* 1–3s later, neither is on the doc at create time. The function sleeps and re-reads three times with waits of **3000, 5000, 6000 ms** (`index.ts:12845–12857`), breaking early once an inviter or campaign lands. An organic signup waits out the full **14 seconds** and burns 3 extra document reads.

- Builds a Hebrew "via" suffix (`index.ts:12863-12888`), precedence: tracked link → personal referral → campaign → bare source:
- `linkName + ' דרך קישור ' / ' דרך קישור ' ( :12874 )`
- `invName + ' דרך ' / ' דרך הזמנה ' ( :12880 , :12882 )`
- `campaign + ' דרך קמפיין ' ( :12885 )`
- `source + ' דרך קישור ' ( :12887 )`
- Sends **to the founder, not to the user** — `pushToAdmins('newUser', 'Teamder', \ מישור נרשם לאפליקציה 🎉` (`{name}}${via}`, { uid })` ( `index.ts:12890-12895` ). `pushToAdmins (/Users/matan/Projects/soccer/functions/src/adminPush.ts:12-...)` honours a per-type mute in `adminConfig/prefs (:22-29)` and a 20-second same-type flood latch ( `:38-49`).

There is no welcome push, no welcome email, and no welcome notification to the new user. `grep -rni "welcome" /Users/matan/Projects/soccer/functions/src` returns nothing.

No server-side Joryio integration exists. `grep -rn "joryio" /Users/matan/Projects/soccer/functions/src` returns nothing — every Joryio identify/track/attribute call in this system is client-side.

4.7 CAMPAIGN ENROLMENT

There is no enrolment step. Campaigns are **evaluated against a segment at send time**, so a new user is eligible from the moment their doc exists.

- **Push campaigns:** `sweepDueCampaigns` (`/Users/matan/Projects/soccer/functions/src/adminUserPush.ts:422-435`) is run from the cron at `index.ts:13480` ; it picks up to 10 queued campaigns whose `sendAt` has passed and calls `processCampaign` (`adminUserPush.ts:231-419`), which **reads the entire** `/users` **collection** (`adminUserPush.ts:311`) and matches each user against the segment.
- Segment fields relevant to a new signup (`adminUserPush.ts:320-344`): `daysSinceJoin` computed as `(now - u.createdAt) / DAY ( :126 )`, `invited: !!u.invitedBy`, `platform: u.platform`, `inGroup`, `hasPush`.
- `hasPush` is **computed from the root** `u.fcmTokens` **field** (`adminUserPush.ts:341`) — which the converter no longer writes (`firestore.ts:244-249`). Actual delivery reads the private subdoc (`tokensFor` , `adminUserPush.ts:206-217`). So a brand-new user who granted push permission is `hasPush: false` for **targeting** while being perfectly reachable for **delivery**. The identical bug exists client-side for popup campaigns at `/Users/matan/Projects/soccer/src/services/campaignService.ts:131`.
- **Per-user daily cap:** anyone who received a broadcast in the last 24h is dropped (`adminUserPush.ts:356`), and `lastBroadcastAt` is stamped on everyone reached (`adminUserPush.ts:396-402`).

4.8 THE JORYIO IN-APP ONBOARDING WIZARD

A **second onboarding**, delivered as a Joryio in-app HTML campaign rather than as app code.

`/Users/matan/Projects/soccer/src/services/onboardingService.ts:1-12` describes it: an HTML document served under `script-src 'none'` — "it can show, it can branch, and it can hold what somebody typed, but it cannot call anything." A single `data-action="submit"` is turned into one call by the shim in `/Users/matan/Projects/soccer/src/components/joryio/InAppMessageHost.tsx:157`.

`submit ( onboardingService.ts:72-118 )`: - `role` is `'player'` or `'organiser'` (`:73`); `recordRole(role)` fires (`:74`). - `clubMode === 'join'` → `groupService.requestJoinByCode(code)` (`:86`), advances to step `w6`. - Otherwise creates a club via `groupService.createGroup` (`:105-110`) and advances to `w5` with a short invite

link (:112). - Error strings: he.onboardingNeedCode (:83), he.onboardingBadCode (:88), he.onboardingNeedClubName (:102), he.error (:77 , :95 , :116).

recordRole (/Users/matan/Projects/soccer/src/services/roleService.ts:23-30) writes the answer **both ways**: logEvent(AnalyticsEvent.RoleSelected, { role }) (:25) and joryio.setAttributes({ user_type: role }) (:26). The comment at roleService.ts:13-22 explains why: "the event is the timestamp... the attribute is the state... Joryio gets no merge fields from us, so anything a future message wants to know has to be on the profile before it is sent."

The only channel an in-app message has is a link, because the document runs under script-src 'none' — roleService.ts:3-6 gives the shape: footy://open/create-community?role=organiser .

This means the "what kind of user are you?" question a consultant will look for is not in the binary. It is authored in the Joryio console as an HTML campaign, and whether a given new user sees it depends on campaign targeting outside this repo.

4.9 ORGANISER SIGNALS

reportOrganiserState — /Users/matan/Projects/soccer/src/services/organiserSignals.ts:54-88 , called from /Users/matan/Projects/soccer/src/store/groupStore.ts:158 . Sets Joryio attributes from the clubs the user **admins** (organiserSignals.ts:60-62) and raises ClubRosterMilestone / ClubBecamePlayable events (:70-78). Milestone high-water marks are kept in AsyncStorage under 'organiser:rosterBest:v1' (organiserSignals.ts:35), so a reinstall re-sends one.

Two limitations stated in the file header (organiserSignals.ts:9-18): the facts are only as fresh as the last time the organiser opened a club screen, and it only ever reports clubs the person **admins** — "being told to grow somebody else's squad is the kind of message that gets an app muted."

A brand-new user has no clubs, so on signup this reports zero-length attribute arrays and fires no milestone.

4.10 ADS

New accounts get an ad-free honeymoon: showAppOpenAdIfAvailable({ accountCreatedAt: currentUser?.createdAt }) (RootNavigator.tsx:189-191), and the gate at /Users/matan/Projects/soccer/src/services/adsService.ts:441-446 suppresses the app-open ad while Date.now() - accountCreatedAt < rcNumber('app_open_new_user_grace_ms') . Default **2 days** — /Users/matan/Projects/soccer/src/services/remoteConfigService.ts:27 .

The app-open ad only runs once membership === 'member' (RootNavigator.tsx:187), so a user with no club never sees it regardless.

4.11 NEVER CALLED

joryio.resetUser exists (joryio.ts:186-189 , exported at joryio.ts:407) but **has no call site anywhere in src/ or App.tsx** . userStore.signOut (userStore.ts:164-187) clears the group, game and chat stores but does not reset the Joryio identity. So after sign-out the install stays bound in Joryio to the previous person, and a second person signing up on the same device is identify -ed over the top of the first.

5. Failure modes

5.1 THE /USERS DOC WRITE FAILS (RULES DENIED, QUOTA, NETWORK)

Google / Apple / email-sign-up / email-sign-in — `userService.ts:193–207` (Google), `:240–251` (Apple), `:285–295` (email sign-in), `:328–338` (email sign-up). All four do the same three things: 1. `logError('createUserDoc', err, { uid, provider, email })` — writes to the `/errors` collection via the buffered logger (`/Users/matan/Projects/soccer/src/services/errorLog.ts:235–275`), which in turn triggers the `onErrorLogged` founder alert (`functions/src/index.ts:13072`). 2. **Sign the user back out** — `signOutFirebase()` wrapped in its own try/catch (`userService.ts:201–205`). The stated reason at `:189–192`: otherwise we leave a Firebase Auth user with no `/users` doc and the next launch crashes on `currentUser.name`. 3. Re-throw.

The user then sees, from `SignInScreen.tsx:88 / :134`: `appAlert(he.error, friendlySignInError(err))`. `friendlySignInError` (`SignInScreen.tsx:158–171`) has no branch for a Firestore permission error, so the message is the catch-all `he.ts:1264` `signInFailed: 'ההתחברות נכשלה. נסה שוב.'`. **The user is told sign-in failed when sign-in in fact succeeded and only the profile write failed** — and because they were silently signed out, retrying looks identical.

Cold-start lazy-create — `userService.ts:148–154`. Different behaviour: logs, and **returns** `null` instead of signing out. `hydrate` (`userStore.ts:115–120`) turns that into `currentUser: null`, and `RootNavigator.tsx:255` renders `AuthStack initialRoute="SignIn"`. A user with a valid Auth session is shown the sign-in screen with **no error message at all**.

5.2 OFFLINE

Cold start, signed in, no network. `getCurrentUser` races the `getDoc` against an **8-second timeout** (`userService.ts:104–109`). The comment at `:96–101` explains: with no offline persistence, an offline `getDoc` hangs forever and the boot awaits it, so the splash would spin indefinitely — "very common for this app: locker rooms / dead zones." On timeout it falls back to the AsyncStorage snapshot (`userService.ts:112–120`), but only if `cached.id === fbUser.uid` (`:116`) — the cross-account guard. If there is no snapshot, it returns `null` → sign-in screen.

`cacheAuthUser` (`userService.ts:879–888`) is what populates that snapshot. The comment at `:870–872` records that before it existed, the key was only ever written under `USE MOCK DATA`, so **the fallback was dead code in production** — every dead-zone cold start bounced a real signed-in user to `SignIn`.

Signing up while offline. The Google chooser will fail; `SignInScreen.tsx:63–70` classifies `code === 'unavailable'` and messages containing `'network' / 'offline'` as *transient* and deliberately does not log them to the error panel (`:71–77`), but does show `he.ts:1270` `signInNetworkError: 'אינן חייבור לאינטרנט' ["no internet connection"]` via `friendlySignInError` (`SignInScreen.tsx:165–169`).

Sign-out while offline. `removeDeviceTokenBeforeAuthTeardown` (`userStore.ts:84–96`) races the token-removal write against a 4-second cap (`userStore.ts:77`). The long comment at `userStore.ts:57–77` explains the stakes: the write is self-only, so it must land before auth is revoked; the previous 1500 ms cap let slow-but-online writes lose the race, leaving the token on the old account and **leaking the previous user's pushes — including lockscreen DM previews — to the next user on the phone**.

5.3 THE USER CANCELS A GOOGLE SIGN-IN

`auth.ts:98–100`: `if (!isSuccessResponse(result)) throw new Error('Sign-in cancelled')`.

SignInScreen.tsx:53–54 detects cancellation three ways: the message contains 'cancel', the code contains 'cancel', or the code is Google's '12501'. Then: - **No error log** (SignInScreen.tsx:71–77). - **logEvent**(AnalyticsEvent.SignInCancelled, { method: 'google' }) (:79). - **No dialog** — SignInScreen.tsx:85–89: "Don't pop an error dialog when the USER cancelled the Google chooser... cancelling isn't a failure." - The spinner clears in **finally** (:90–92) and the user is back on the sign-in screen with nothing said.

he.signInCancelled: 'ההתחברות בוטלה' exists at he.ts:1262 and is returned by friendlySignInError (SignInScreen.tsx:162) — but the dialog is suppressed for exactly the cancelled case, so **this string is effectively unreachable on the Google path**.

Apple cancellation: auth.ts:360–366 maps ERR_REQUEST_CANCELED / ERR_CANCELED to the same 'Sign-in cancelled'; SignInScreen.tsx:106–107 detects it; same silent handling (:126–135).

Guest failure: no cancellation concept; any error shows he.signInFailed (SignInScreen.tsx:151).

5.4 THE NAME IS REJECTED

Covered in §3.5. In summary: on PostSignInOnboardingScreen — the screen every new user sees — a rejected name produces 'שגיאה' / 'ההתחברות נכשלה. נסה שוב.' (PostSignInOnboardingScreen.tsx:127), which does not describe the problem. The specific messages exist (he.ts:640–643) and are wired only on the legacy ProfileSetupScreen (ProfileSetupScreen.tsx:44–51). If the client check is somehow bypassed, firestore.rules:273–274 denies the write and the user gets a raw permission error through the same generic path.

5.5 EMAIL-PATH FAILURES

handleAuthError — EmailAuthScreen.tsx:89–135:

Firestore code	Shown	he.ts
auth/invalid-email	'כתובת מייל לא תקינה'	:1827
auth/weak-password	'הסיסמה חייבת להכיל לפחות 6 תווים'	:1828
auth/wrong-password, auth/user-not-found, auth/invalid-credential	'מייל או סיסמה שגויים'	:1829
auth/too-many-requests	'יותר מדי ניסיונות. נסה שוב בעוד כמה דקות.'	:1830
auth/network-request-failed	'אין חיבור לאינטרנט'	:1270
anything else	'משהו השתבש, נסה שוב' + logError	:1831, EmailAuthScreen.tsx:131

auth/email-already-in-use gets a two-button dialog (EmailAuthScreen.tsx:109–118): body he.ts:1832–1833 emailAuthAlreadyInUse: 'אם נרשמת עם מייל וסיסמה — עבור להתחברות. אם', buttons 'בטל' (he.ts:52) and 'עבור להתחברות' (he.ts:1834).

There is a *better* message that usually cannot fire. `signUpWithEmail` (`auth.ts:176–196`) tries `fetchSignInMethodsForEmail` to detect a social-provider collision and throw `EmailRegisteredWithProviderError`, which maps to `he.ts:1835`. Google's comment `הכתובת הזו כבר רשומה דרך Google` or `התחבר עם Google` or `he.ts:1836` (Apple). But the comment at `EmailAuthScreen.tsx:110–113` states plainly: **"Email-enumeration protection makes the provider undetectable (`fetchSignInMethodsForEmail` returns `[]`)"** — so in practice the generic `emailAuthAlreadyInUse` is what users get.

Password reset (EmailAuthScreen.tsx:137–166) deliberately reports success for user-not-found so account existence is not leaked (:161–163): he.ts:1837 emailAuthResetSentTitle: 'נשלח מייל איפוס' + he.ts:1838–1839 emailAuthResetSentBody: (email) => \ שלחנו קישור לאיפוס סיסמה אל \${email}. בדוק את תיבת הדואר. (וגם ספאם). Tapping reset with an invalid address: he.ts:1840 emailAuthResetNeedEmail: 'הקלד קודם את' הקלד קודם את' . 'כתובת המייל שלך'.

Client-side gates before submit: EMAIL_RE = /^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)\$/ (EmailAuthScreen.tsx:37), password ≥ 6 (:54), confirm must match on sign-up (:57–60), mismatch shown inline as he.ts:1821 'הסיסמאות לא תואמות' (:258–260).

5.6 PHOTO UPLOAD FAILURE DURING ONBOARDING

PostSignInOnboardingScreen.tsx:63-96 . On `res.reason === 'cancelled' | 'permission'` the screen says **nothing at all** (:73-77) — the comment cites App Store guideline 5.1.1(iv): do not nag or point to Settings.

`'network'` → he.ts:2816 `profilePhotoUploadFailed: 'העלאת התמונה נכשלה. נסה שוב.'`; `'unavailable'` → he.ts:2818-2819 `profilePhotoUnavailable: 'בחירת תמונה לא זמינה כרגע. בחר אוטאר מוכן'`

he.ts:2814-2815 `profilePhotoPermissionDenied: 'אין הרשאה לגישה לגלריה. אפשר לאשר.'` .בינתיים

`'exists'` exists but is **not used on this screen**.

5.7 BOOT HYDRATION FAILURE

hydrate (userStore.ts:103-123) wraps each of its two reads so neither can leave hydrated: false forever — the comment at :104-108 notes RootNavigator gates the splash on that flag and "silent rejections meant a perma-splash that was unrecoverable without a force-close." Failures log BootHydrateFailed boot_hydrate_failed (analyticsService.ts:328) with source: 'storage' or source: 'user_read' (userStore.ts:113 , :117).

6. The `onboardingCompleted` flag

6.1 THERE ARE TWO SEPARATE ONBOARDING FLAGS

They are frequently confused. Both are read by `RootNavigator`.

	onboardingDone	onboardingCompleted
Where it lives	AsyncStorage, key <code>'footy.onboarding.done'</code> (<code>/Users/matan/Projects/soccer/src/services/storage.ts:7</code>)	Firestore field on <code>/users/{uid}</code>
Scope	per device install	per account
Read by	<code>userStore.ts:110</code> , <code>RootNavigator.tsx:35</code>	<code>userStore.ts:248–251</code> , <code>RootNavigator.tsx:38</code>

	onboardingDone	onboardingCompleted
Set by	<code>userStore.completeOnboarding → storage.setOnboardingDone(true)</code> (<code>userStore.ts:125–128</code> , <code>storage.ts:88–90</code>)	<code>userService.completeOnboarding</code> (<code>userService.ts:394</code>)
Gates	the pre-sign-in slides (<code>RootNavigator.tsx:253</code>)	the post-sign-in profile screen (<code>RootNavigator.tsx:267–269</code>)
Survives reinstall?	No	Yes
Survives account switch on one device?	Yes (device-scoped)	No

6.2 WHO SETS ONBOARDINGCOMPLETED

Written `false` **on every create** — via the converter at `firebase.ts:191` (`u.onboardingCompleted ?? false`), from the `fresh` objects at `userService.ts:187` , `:238` , `:283` , `:326` , `:146` .

Set `true` **in exactly one place:** `userService.completeOnboarding` (`userService.ts:355–419`). The update at `userService.ts:392–396` is:

```
{ name: trimmedName, onboardingCompleted: true, updatedAt }
```

plus `avatarId` if given (`:397`) and `photoUrl` if given (`:398`). Called only from `userStore.completePostSignInOnboarding` (`userStore.ts:253–258`), which is called only from `PostSignInOnboardingScreen.handleSave` (`PostSignInOnboardingScreen.tsx:116–120`).

So: **name and** `onboardingCompleted` **are set in the same single write**. There is no partial state where a user has a name but has not "completed onboarding" — except for legacy accounts that pre-date the field.

Read back with a strict equality check: `onboardingCompleted: d.onboardingCompleted === true` (`firebase.ts:278`) and `u.onboardingCompleted === true` (`userStore.ts:250`). Missing or null reads as `false` .

Because `updateDoc` bypasses Firestore converters, this write sends exactly those four keys — it does not re-serialise the whole user object.

6.3 WHAT IS GATED ON IT

1. `PostSignInOnboardingScreen` — `RootNavigator.tsx:267–269` : `if (!isGuest && currentUser && !hasCompletedOnboarding) return <PostSignInOnboardingScreen />;` This is a **full-screen block with no back button, no skip and no dismiss**. There is no route out of it except saving a valid name. Guests bypass it entirely via the `isGuest` term (`RootNavigator.tsx:260` , and `buildGuestUser` hard-codes `onboardingCompleted: true` at `userService.ts:50`).
2. `ProfileSetupScreen` — `RootNavigator.tsx:271` , reached only when onboarding is complete but the name is empty. Effectively legacy-only (see §3.5).
3. **The pending-invite / deep-link consumer** — `RootNavigator.tsx:65` : `if (!currentUser || !profileComplete || !hasCompletedOnboarding) return;` **A user who arrived from an invite link to a specific game does not reach that game until they have finished onboarding**. The stash survives (`RootNavigator.tsx:68`), the navigation is simply deferred until all three conditions hold, then pre-flighted for

existence (:95–115) and navigated (:146–157). If the target no longer exists the user gets a toast: 'הקישור לא תקין או שהפריט כבר לא קיים' (RootNavigator.tsx:124) plus an InviteLinkDead invite_link_dead event (:120–123 , analyticsService.ts:329).

isProfileComplete (userStore.ts:243–246) is the independent second condition: !!u && u.name.trim().length > 0 .

Grandfathering is deliberate. RootNavigator.tsx:262–266 : "Existing accounts that never had this field stay grandfathered as long as the converter writes false rather than promoting them — they'll see it once."

7. Order and timing — a complete trace of one Google signup on a fresh install

T ∞ — process start, before any UI. /Users/matan/Projects/soccer/App.tsx:17 void joryio.init() fires at module scope, before ExpoSplash.preventAutoHideAsync() at :19 . initJoryio (joryio.ts:103–118) starts the SDK with the platform key (joryio.ts:39–55 , literals present as fallbacks because an env-only key resolved to '' in a store build and shipped analytics silently off — joryio.ts:31–37).

T0 — boot. RootNavigator mounts; hydrateUser() + initRemoteConfig() + adsService.initializeAds() (RootNavigator.tsx:170–181). hydrate reads onboardingDone and calls getCurrentUser in parallel (userStore.ts:109–121). getCurrentUser → waitForAuthRestore() (userService.ts:86) registers the onAuthStateChanged listener (auth.ts:458); first emission is null , promise resolves, **listener stays alive** (auth.ts:456–474). Returns null .

T0+ — deep-link / attribution resolution, in parallel. App.tsx:537–577 , strictly ordered: Linking.getInitialURL() (:539) → if a pending invite already exists, stop (:545–547) → Android Play Install Referrer consumeInstallReferrerIfFresh() (:553) → iOS clipboard deferred deep link consumeClipboardInviteIfFresh() (:569). Whatever lands is stashed under PENDING_INVITE in AsyncStorage, shape at storage.ts:75–81 .

T1 — pre-sign-in slides. RootNavigator.tsx:253 renders OnboardingScreen while onboardingDone is false. (Not this surface.)

T2 — SignInScreen. RootNavigator.tsx:255 .

T3 — user taps 'Google עם המטד'. logEvent(SignInAttempted, { method:'google' }) (SignInScreen.tsx:45) → setBusyProvider('google') → spinner replaces the button content (:199–206).

T4 — native picker. ensureGoogleConfigured() (auth.ts:90 , config :59–68); Android-only hasPlayServices (auth.ts:93–95); GoogleSignIn.signIn() (auth.ts:97).

T5 — Firebase credential exchange. signInWithCredential (auth.ts:110) → mirrorToNativeAuth fire-and-forget (auth.ts:112).

T5a — in parallel, the surviving listener fires. ensureNativeAuthMirror() (auth.ts:462) then joryio.identify(uid, { email, name }) (auth.ts:468–471), then unsub() (auth.ts:473).

T6 — one read. getDoc(docs.user(uid)) (userService.ts:175). Does not exist.

T7 — one write. setDoc(ref, converter(fresh)) (userService.ts:194). **This is the moment onNewUserJoined starts.**

T8 — attribution, awaited. `applyInviteAttributionIfFresh ( userService.ts:208 )`: AsyncStorage read + Firestore read + possible write. Then `applyAcquisitionIfFresh ( :209 )`: AsyncStorage read + Firestore read + possible write. Then `cacheAuthUser ( :210 )`.

T9 — store update. `set({ currentUser: user }) ( userStore.ts:132 )`, `logEvent(SignInSuccess) ( :133 )`.

T10 — the user sees the next screen. `RootNavigator` re-renders. `hasCompletedOnboarding` is `false` → `PostSignInOnboardingScreen ( RootNavigator.tsx:267–269 )`.

T10a — same tick, background effects. - `subscribeCurrentUser(uid)` attaches a live `onSnapshot` on `/users/{uid}` (`RootNavigator.tsx:226–229`, `userStore.ts:207–224`). - `hydrateGroup / setGameCurrentUserId / hydratePlayers / subscribeGroups ( RootNavigator.tsx:201–218 )`. - `requestAndRegisterPushToken ( RootNavigator.tsx:241–247 )` → the **OS push-permission dialog appears on top of the onboarding screen**.

T10b — server side, 3 to 14 seconds. `onNewUserJoined` sleeps 3s, reads; 5s, reads; 6s, reads (`index.ts:12845–12857`); builds the `via` suffix; pushes `'(...) 🎉! מִיִּשְׁהוּ נִרְשָׁם לֹאֲפִלִּיקָצִיָּה'` to the founder's Pulse devices (`index.ts:12890–12895`).

T11 — the user types a name and taps `'המשיך'` (`he.ts:2840` `psProfileSave: 'המשיך'`). `complete({ name, avatarId, photoUrl }) ( PostSignInOnboardingScreen.tsx:116–120 )` → `completeOnboarding` → `sanitize ( userService.ts:363 )` → `reserved check ( :368 )` → `email-shape check ( :373 )` → `one updateDoc ( :400 )` → `cacheAuthUser ( :417 )` → `ProfileCreated` + `OnboardingCompleted` events (`userStore.ts:256–257`).

T12 — the app opens. `RootNavigator.tsx:274` waits on `groupHydrated`, then `MainTabs ( :279 )`. No dedicated "you have no club" screen exists — the comment at `RootNavigator.tsx:275–278` says both that state and "pending request" fall through to `MainTabs` and surface inline.

T13 — deferred deep link, once. `consumedRef` fires the pending-invite consumer exactly once per launch (`RootNavigator.tsx:62–66`).

Minimum required interactions from a cold Google install to `MainTabs`: tap `'Google עם המשיך'` → pick an account in the OS sheet → answer the OS push dialog → type a name → tap `'המשיך'`. Five, of which two are OS dialogs and one is free-text entry. Everything else on `PostSignInOnboardingScreen` — photo upload, the 24-avatar grid — is optional.

8. Branch matrix — what differs by state

Branch	Where decided	Behaviour
Android vs iOS — provider set	<code>SignInScreen.tsx:212</code>	Apple button renders on iOS only. Android users have Google / email / guest.
Android vs iOS — Apple availability	<code>auth.ts:310–317</code> , <code>auth.ts:338–340</code>	<code>isAppleSignInAvailable</code> returns false off iOS; <code>signInWithApple</code> throws 'Apple Sign-In is only available on iOS.'. <code>isAppleSignInAvailable</code> is not called by <code>SignInScreen</code> — the button is shown on all iOS versions rather than gated on iOS 13+.
Android vs iOS — Play Services	<code>auth.ts:93–95</code>	<code>hasPlayServices({ showPlayServicesUpdateDialog: true })</code> on Android only.
Android vs iOS — install attribution	<code>App.tsx:549–576</code>	Android: Play Install Referrer. iOS: clipboard deferred deep link (Apple has no referrer API).

Branch	Where decided	Behaviour
Android vs iOS — native auth mirror	auth.ts:225 , auth.ts:283	Android only; feeds the home widget and Wear relay.
Android vs iOS — Google sign-out on signOut	auth.ts:418	GoogleSignIn.signOut() runs on Android only, so the next iOS sign-in may silently re-use the cached account.
Guest vs registered	RootNavigator.tsx:260	Guest: no /users doc, no push prompt (:240), no onboarding screen (:267), no profile gate (:271) — straight to MainTabs .
Has name from provider vs not	userService.ts:183 / :234 / :322	Google pre-fills the name field (PostSignInOnboardingScreen.tsx:40 seeds from user?.name); Apple pre-fills on first authorization only; email sign-up always starts empty (:322 hard-codes '').
First Apple authorization vs later	auth.ts:326–329 , userService.ts:230–233	Apple returns fullName only once, ever. A user who deletes the account and signs in again with Apple gets no name.
New account vs existing doc	userService.ts:176 / :223 / :272 / :315	If the doc already exists, the existing one is returned and no attribution or acquisition is applied — correct for a returning user, and the reason re-installs do not re-attribute.
Email sign-up vs email sign-in	userService.ts:339–340 vs :296	Sign-up runs both attribution helpers; sign-in does not.
Has pending invite vs organic	userService.ts:914 , :954	Organic signups skip both writes entirely, and onNewUserJoined waits the full 14s for nothing (index.ts:12845–12857).
Has club vs no club	RootNavigator.tsx:187 , organiserSignals.ts:60	No club → no app-open ad; no organiser attributes; no milestone events.
Online vs offline cold start	userService.ts:104–122	8s race; falls back to the AsyncStorage snapshot, else SignIn.
Legacy account (no onboardingCompleted)	firestore.ts:278 , RootNavigator.tsx:262–266	Reads as false ; sees PostSignInOnboardingScreen exactly once.
Account with an empty name but onboardingCompleted : true	RootNavigator.tsx:271	Only path to ProfileSetupScreen , the one screen with correct name-rejection copy.
Mock mode (USE MOCK DATA)	userService.ts:62 , :168 , :215 , :264 , :307	Every path returns mockCurrentUser and writes AsyncStorage only. Attribution, presence, analytics delivery (analyticsService.ts:579) and Joryio (joryio.ts:104 , :124 , :134) are all no-ops. Mock QA cannot exercise any of this surface.

Branches I did not check: web (Platform.OS === 'web') — auth.ts:83–87 throws for any platform other than android/ios and auth.ts:9 notes web Google sign-in is "not yet wired"; I did not verify whether the app is ever built for web. I also did not check Expo Go behaviour beyond noting the explicit bail-outs at

9. Facts a consultant will ask about that are worth stating plainly

1. **No terms-of-service or privacy-policy link exists on the sign-in screen.** `he.ts:1799` `באמצעות התחברות אתה` `'מסכים לתנאי השימוש'` is rendered as inert text at `SignInScreen.tsx:272` .
2. **No pre-permission priming for push.** The OS dialog is the first and only thing said about notifications, and it lands mid-onboarding (`RootNavigator.tsx:235–248`).
3. `marketingPush` **defaults to** `true` (`types/index.ts:423`) and is never surfaced during signup.
4. **The Google profile photo is discarded** — `photoUrl` is absent from the converter's write path (`firestore.ts:183–264` , note at `:274`).
5. **Every new user is a random emoji until they act** (`userService.ts:185` + `avatars.ts:70–72`).
6. **Every signed-in user can read every user document** (`firestore.rules:260`).
7. **The founder gets a push for every signup; the user gets nothing** (`index.ts:12890–12895` ; no welcome anything anywhere in `functions/src`).
8. `createdAt` **is the device clock** (`userService.ts:185` , `firestore.ts:189`), and `daysSinceJoin` segmentation is computed from it (`adminUserPush.ts:126`). A phone with a wrong clock lands in the wrong segments.
9. `hasPush` **segmentation is broken for all new users** — it reads a root field the converter no longer writes (`adminUserPush.ts:341` vs `firestore.ts:244–249` ; same at `campaignService.ts:131`).
10. **Joryio is never reset on sign-out** (`joryio.ts:186–189` has no caller), so shared devices cross-contaminate the marketing profile.
11. **The "are you an organiser or a player?" question is not in the app.** It is a Joryio in-app HTML campaign (`onboardingService.ts:1–12` , `roleService.ts:3–6`), so whether a new user is asked at all is decided outside this repo.
12. **The name-rejection copy on the screen everyone sees is wrong** (`PostSignInOnboardingScreen.tsx:127`), and the correct copy exists two files away (`ProfileSetupScreen.tsx:44–51` , `he.ts:640–643`).

Open questions from this section

Things this reader could not settle from the code, listed rather than assumed.

- Does the Joryio in-app onboarding wizard actually fire for new users in production, and on which trigger? The renderer (`InAppMessageHost.tsx:157`) and the handler (`onboardingService.ts:72–118`) are in the binary, but the campaign, its segment and its HTML live in the Joryio console. Everything about when a new user meets that wizard — or whether they ever do — has to be read there, not here.
- Is the Google Play pre-launch report still enabled? `officialAccount.ts:80–85` and `firestore.rules:179–181` both state plainly that the name checks stop the robot completing a profile but do NOT stop it running — only unticking the pre-launch report in Play Console does. Needs a Play Console check, plus a count of accounts created after 19.09.2026 carrying an '@'.
- What does the Google account chooser actually look like on a device with zero Google accounts signed in, and what error code comes back? `SignInScreen.tsx:63–70` classifies a long list of codes as transient, but I could not confirm which code that specific state produces or whether the user gets a usable message.

- Exact wall-clock cost of the awaited attribution block (userService.ts:208-209) on a real cellular connection. It is two Firestore reads plus up to two writes between the Google sheet closing and the next screen appearing; the code cannot tell me whether that is 300ms or 3s in the field.
 - Whether the OS push-permission dialog on iOS lands before or after PostSignInOnboardingScreen has painted. The effect order in RootNavigator.tsx:235-248 says 'same tick as the render', but the actual visual sequence — and whether the user sees the dialog over a blank screen or over the hero — needs a device recording.
 - Whether Apple's private-relay addresses appear in email for Apple signups in production, and what share of Apple accounts therefore have a relay address on the /users doc (userService.ts:235).
 - Whether emailVerified is enforced anywhere downstream. auth.ts:171 sends the verification mail and auth.ts:133-135 says usage is not blocked on it; I found no read of the flag in src/, but I did not audit functions/src for one.
 - Whether any real user has ever completed the email sign-up path. It is the only path that hard-codes name: " (userService.ts:322), so it is also the one with the emptiest first impression — a usage split by provider from Firebase Auth would settle whether this matters.
 - The real-world impact of the hasPush segmentation gap (adminUserPush.ts:341 reading a field firestore.ts:244-249 no longer writes): how many queued campaigns have used a hasPush rule, and how many users they wrongly excluded. Needs a query over /campaigns in production.
 - Whether latestVersion / soft-update gating interacts with first launch for a brand-new install. Out of my reading scope here, but it is another thing that can draw a modal over the onboarding screen.
-

5.4 · From a fresh account to a registered game

Surface: from a fresh account to registered for a game

Everything below is read from source at /Users/matan/Projects/soccer . Every Hebrew string is quoted verbatim with its src/i18n/he.ts key and line. Nothing was observed on a device — see Open Questions.

0. Where a brand-new signed-in user lands

RootNavigator is the decider, in this order (src/navigation/RootNavigator.tsx:251–279):

1. if (!userHydrated) return <Splash /> — :251
2. if (!onboardingDone) return <OnboardingScreen /> — :253
3. if (!currentUser) return <AuthStack initialRoute="SignIn" /> — :255
4. const isGuest = currentUser.isGuest === true — :260
5. if (!isGuest && currentUser && !hasCompletedOnboarding) return <PostSignInOnboardingScreen /> — :267
6. if (!isGuest && !profileComplete) return <AuthStack initialRoute="ProfileSetup" /> — :271
7. if (!groupHydrated) return <Splash /> — :274
8. return <MainTabs /> — :279

Explicit comment, `src/navigation/RootNavigator.tsx:275-278`: "No more dedicated full-screen views for 'pending request' or 'new user without community'. Both states fall through to MainTabs and surface their context inline (toasts on submit + a 'pending' tag in the communities feed)."

The landing tab is the Profile tab, which is "home". `initialRouteName="ProfileTab"` at `src/navigation/MainTabs.tsx:110`; `ProfileTab` renders `ProfileStack` whose configured root is `Profile` (`src/navigation/MainTabs.tsx:190`), i.e. `src/screens/tabs/ProfileScreen.tsx`.

Tab registration order (`src/navigation/MainTabs.tsx:140-186`): `ProfileTab`, `CommunitiesTab`, `GameTab`, `ChatTab`. Index 0 is visual RIGHT under forceRTL, so right→left the bar reads `בית · מועדונים · מחזורים · צ'אטים`.

key	value	line
tabHome	'בית'	src/i18n/he.ts:2887
tabCommunities	'מועדונים'	src/i18n/he.ts:1273
tabGame	'מחזורים'	src/i18n/he.ts:2885
tabChat	'צ'אטים'	src/i18n/he.ts:4049

⚠ The file's own header comment (`src/navigation/MainTabs.tsx:27-30`) describes a **different, 3-tab** layout — "right: Communities → center: Games (primary) → left: Profile" — which contradicts the code below it. The comment is stale; the code is the truth.

Branch not checked: the guest (anonymous) session bypasses steps 5 and 6 entirely (`src/navigation/RootNavigator.tsx:257-271`), and `ProfileScreen` renders a completely different screen for a guest (see §1.9). I did not verify how a guest is created (which button starts an anonymous session) — that is the auth/onboarding surface.

1. The home screen for a user with NO club and NO games

File: `src/screens/tabs/ProfileScreen.tsx`. This is the single most important screen in the report, so every block is listed in render order with its render condition.

1.1 WHAT DATA THE SCREEN FETCHES ON FOCUS, AND WHAT IT RETURNS FOR AN EMPTY ACCOUNT

All of these are `useFocusEffect` hooks that **short-circuit to empty when there is no club** — this is what makes the empty home empty:

state	fetch	empty-account result	line
nextGame , myGames , createdGames	<code>gameService.getMyLiveOrUpcomingGames(uid)</code>	<code>null / []</code>	:294 -336
justPlayed	<code>gameService.getJustFinishedGame(uid)</code>	<code>null</code>	:337 -344
openToJoin	<code>gameService.getCommunityGames(uid, myCommunities.map(...))</code> — guarded by <code>myCommunities.length === 0</code> → <code>setOpenToJoin([])</code>	<code>[]</code>	:344 -372

state	fetch	empty-account result	line
scheduledUpcoming	gameService.getMyUpcomingScheduledGames(...) — same club guard	[]	:376 -399
inboxCount	getInboxCount(uid)	0	:402 -422
availData	availabilityFeedService.getAvailabilityCounts()	see §1.6	:436 -453
playedThisWeek, lastPlayedMs	gameService.getPlayedGames(uid, 20)	0, null	:456 -486
playedCount	gameService.getPlayedGamesCount(uid)	0	:489 -512
clubInsight	assistantInsightsService.getClubInsight(...) — guarded by !groupId	null	:519 -542
referralCount, referrals	userService.listInvitedUsers(uid)	0, []	:260 -291

Hero selection is a pure helper: `const heroPick = pickHomeHero<Game>([], openToJoin, scheduledUpcoming)` (`src/screens/tabs/ProfileScreen.tsx:167`). With no club both arrays are empty, so `pickHomeHero` returns `{ kind: 'none', game: null }` (`src/utils/homeHero.ts:34-39`).

1.2 ① TOP BAR — ALWAYS RENDERED

`<HomeTopBar />` at `src/screens/tabs/ProfileScreen.tsx:1152-1168` . Component: `src/components/home/HomeDashboardParts.tsx:27-67` .

- Visual RIGHT: the user's avatar + a `chevron-down` (:43-46), tap → `nav.navigate('ProfileEdit')` (`ProfileScreen.tsx:1167`).
- Visual LEFT: `menu` icon → opens the hamburger (:50-52), and `notifications-outline` with a red dot when `inboxCount > 0` (:53-56), tap → `nav.navigate('Requests')` (`ProfileScreen.tsx:1157-1164`).
- Absolutely-centred logo (`src/assets/images/logo.png` , :23) + text.
- `homeBrandName: 'Teamder'` — `src/i18n/he.ts:2019`

For a brand-new user the bell dot is **off** (`inboxCount` is 0).

1.3 ② "הודעה מהמאמן" — THE ASSISTANT CARD. THIS IS THE ONLY WELCOME COPY.

`<AssistantCard message={assistantMessage} greeting={coachGreeting} onCta={handleAssistantCta} />` — `src/screens/tabs/ProfileScreen.tsx:1173-1178` . Component `src/components/home/AssistantCard.tsx:27-68` .

Card header: `assistantTitle: 'הודעה מהמאמן'` — `src/i18n/he.ts:3099` . Line rendered = ``${he.assistantGreeting(greeting)} ${message.text}`` (`AssistantCard.tsx:40-42`), where `assistantGreeting: (greeting) => \ ${greeting},`` — `src/i18n/he.ts:3101`` .

Greeting word by device clock (`ProfileScreen.tsx:769-775`) : - `greetingMorning: 'בוקר טוב'` (05:00–11:59) — `src/i18n/he.ts:3073` - `greetingNoon: 'צהריים טובים'` (12:00–16:59) — :3074 - `greetingEvening: 'ערב טוב'` (17:00–21:59) — :3075 - `greetingNight: 'לילה טוב'` (22:00–04:59) — :3076

`coachGreeting = firstName ? \ ${greetWord} ${firstName}` : greetWord – ProfileScreen.tsx:784`.`

Which rule fires for a brand-new user with no club. The resolver picks the single lowest-priority-number message (`src/Utils/assistant/resolve.ts:68–88`); ties break on registration order in `ASSISTANT_RULES` (`src/Utils/assistant/rules.ts:769–779`). Priority bands: `src/Utils/assistant/types.ts:34–51`.

For an account with 0 clubs, 0 games, no availability, no stats: - `gameDayRule` → null (no `nextGame`) — `rules.ts:115` - `postGameRule` → null (`lastPlayedMs == null`) — `rules.ts:174` - `availabilityMatchRule` → null (no `bestEvening`) — `rules.ts:223–224` - `comebackRule` → null — `rules.ts:247` - `clubIdleRule` → null (`ctx.communities.length === 0`) — `rules.ts:273` - `statsRule` → depends on whether the user picked a preferred position during profile setup. `statsCandidates` pushes a `position` candidate whenever `ctx.user?.position` is set (`rules.ts:632–638`). If it is, `statsRule` fires at `STATS: 5` and **outranks the welcome line**, and the coach's first ever sentence becomes `he.assistantPositionLine(POSITION_LABEL[position])` plus a `'לסטטיסטיקה'` CTA (`assistantStatsCta`, `src/i18n/he.ts:3268`) pointing at an empty stats screen. If `position` is unset, `statsCandidates` returns `[]` and `statsRule` returns null (`rules.ts:663–671`). - `joinClubRule` → fires at `DISCOVERY: 6` — `rules.ts:704–717`.

`joinClubRule` output: - `assistantJoinClub` (one drawn per screen mount, `rules.ts:710`) — `src/i18n/he.ts:3288–3291`: - `'🎲 יש כדורגל מסביבך' - '👋 !Teamder-ל הבא ברוך הבא ל-Teamder' - sub:`
`assistantJoinClubSub`: `'🎲 הגיע הזמן למצוא את החברה שלך' — src/i18n/he.ts:3292` - CTA:
`assistantJoinClubCta`: `'גלה מועדונים' — src/i18n/he.ts:3293` → `{ kind: 'discoverClubs' }` → `nav.navigate('CommunitiesTab')` (`ProfileScreen.tsx:924–926`)

So the full first line is one of: - `'🎲 יש כדורגל מסביבך' - '👋 !Teamder-ל הבא ברוך הבא ל-Teamder' - sub:`
`'🎲 הגיע הזמן למצוא את החברה שלך' — src/i18n/he.ts:3292` - CTA:
`'גלה מועדונים' — src/i18n/he.ts:3293` → `{ kind: 'discoverClubs' }` → `nav.navigate('CommunitiesTab')` (`ProfileScreen.tsx:924–926`)

The draw is random per mount: `newAssistantNonce()` is `Math.floor(Math.random() * 0xffffffff)` (`src/Utils/assistant/resolve.ts:56–58`), minted once per mount (`ProfileScreen.tsx:622`). **The welcome greeting is a coin flip** — half of first-time users never see `'👋 !Teamder-ל הבא ברוך הבא ל-Teamder'` at all.

`markAvailabilityRule` (`rules.ts:722–737`) also sits at `DISCOVERY: 6` but is registered *after* `joinClubRule`, and it self-suppresses anyway because `ctx.shown.availabilityPrompt` is true for this user (`rules.ts:725`; the flag is computed at `ProfileScreen.tsx:875–876`).

If `message` is null the card renders nothing at all (`AssistantCard.tsx:38`).

1.4 THE "JUST PLAYED" CARD — NOT SHOWN

`justPlayed ? ... : null` at `ProfileScreen.tsx:1185–1215`. Never fires for a new account. Strings (unreachable here, listed for completeness): - `homeJustPlayedTitle`: `'המחזור האחרון' — src/i18n/he.ts:718` - `homeJustPlayedBody`: `'המחזור הסתיים - הציון, הגולים והדירוג שלך מחכים' — src/i18n/he.ts:719` - `homeJustPlayedCta`: `'לפרטי המחזור' — :722`

1.5 ③ THE HERO SLOT — RENDERS NOTHING

`{nextGame ? <HomeNextGameCard .../> : heroGame ? <UpcomingScheduledGameCard .../> : null}` — `src/screens/tabs/ProfileScreen.tsx:1228–1259`.

For a new user both are null, so **the hero slot renders literally nothing**. The screen jumps from the coach's line straight to the three action tiles.

⚠ **Dead empty state.** `HomeNextGameCard` contains a fully-built empty state (`src/components/home/HomeNextGameCard.tsx:42–61`) with `homeNoGameTitle`: 'אין לך מחזור קרוב' (`src/i18n/he.ts:2026`), `homeNoGameBody`: 'מצא מחזור פתוח או פתח מחזור חדש' (`:2027`) and a button `homeNoGameCta`: 'מצא מחזור' (`:2028`). It is **unreachable**: the only caller in the app is `ProfileScreen.tsx:1230` , and it is inside a `nextGame ? ...` guard, so `game` is never null when the component mounts (`grep -rn "HomeNextGameCard" src` returns only that one call site plus two unrelated comments). The "no upcoming game" copy that was written for this screen never renders.

1.6 ④ RECOMMENDED-DAY BANNER — HIDDEN FOR A NEW USER

`{availCardEnabled && recommended ? <HomeRecommendedDay .../> : null}` — `ProfileScreen.tsx:1267–1294` .

`recommended` requires `availReady = !!availData && !availData.error && availData.hasLocation && availData.days.length > 0` (`ProfileScreen.tsx:803–807`), and then at least one evening with `count > 0` (`:816–818`). `availData` comes from the `availabilityCounts` callable (`src/services/availabilityFeedService.ts:100–110`); on any error it returns `{ radiusKm: 0, hasLocation: false, days: [], error: true }` (`:124`). A brand-new account has no saved home city, so `hasLocation` is expected false → banner hidden.

Strings (not shown for a new user): `homeRecommendedTitle`: 'היום המומלץ לפתיחת מחזור' (`src/i18n/he.ts:2046`), `homeRecommendedLine`: (day, n) => \ יום \${day} • \${n} 2047: ('פנויים').

`availCardEnabled` is a Pulse/server master switch read on focus, defaulting `true` (`ProfileScreen.tsx:176–177` , `:425–434`).

1.7 ⑤ THREE ACTION TILES — ALWAYS RENDERED, UNCONDITIONAL

`<HomeActionTiles onOpen onAvailability onJoin />` — `ProfileScreen.tsx:1297–1324` . Component: `src/components/home/HomeDashboardParts.tsx:127–143` . Rendered right-to-left in that array order.

tile	label	he.ts	destination
1 (right)	<code>homeActionOpenTitle</code> : 'פתח מחזור'	<code>:2049</code>	<code>nav.navigate('GameTab', { screen: 'GamesList', params: { openCreate: true } })</code> — <code>ProfileScreen.tsx:1303–1306</code>
2	<code>homeActionAvailTitle</code> : 'סמן זמינות'	<code>:2050</code>	<code>nav.navigate('AvailabilityEdit')</code> — <code>:1313</code>
3 (left)	<code>homeActionJoinTitle</code> : 'הצטרף למחזור'	<code>:2051</code>	<code>nav.navigate('GameTab')</code> — <code>:1321</code>

There are no subtitles; the comment at `src/components/home/HomeDashboardParts.tsx:145–148` says they were removed because "הצטרף למחזור" was clipping in a ~110pt column.

1.8 ⑥ AVAILABILITY PODIUM OR THE AVAILABILITY PROMPT CARD

`{availCardEnabled && podium.length > 0 ? <HomeAvailabilityWindows/> : !markedAvailability ? <AvailabilityPromptCard/> : null}` — `ProfileScreen.tsx:1326–1366` .

New user → `podium` empty (§1.6) and `markedAvailability` false (`ProfileScreen.tsx:798–799`), so the **prompt card renders**. Component `src/components/home/AvailabilityPromptCard.tsx:13–34` :

- `availFeedPromptTitle`: 'רוצה לראות מי פנוי לשחק לידך?' — `src/i18n/he.ts:2749`

- `availFeedPromptBody`: 'הגדר את האזור והזמנים שנוח לך - ונראה לך כמה שחקנים פנויים בכל חלון, כדי לפתוח מחזור בקלות' — :2750-2751
- `availFeedPromptCta`: 'הגדר זמינות' — :2752 → `nav.navigate('AvailabilityEdit')` (`ProfileScreen.tsx`:1358-1362)

Podium strings (not shown here): `homeWindowsTitle`: 'פנויים לידך' (:2053), `homeWindowsPlayersUnit`: 'שחקנים' (:2054), `homeWindowsShowWeek`: 'הצג שבוע מלא' (:2055), `homeDayLabel`: (`l`) => \ יום \${l} 2057: (``).

1.9 ⑦ THE ACTIVATION CHECKLIST — "בוא נתחיל"

`{homeDataReady && !checklistComplete ? <OnboardingChecklist items={checklistItems} /> : null}` — `ProfileScreen.tsx`:1370-1372 .

`homeDataReady = groupsHydrated && playedCount !== null && (localUser?.isGuest || referralCount !== null)` (`ProfileScreen.tsx`:719-722) — deliberately delays the card past first paint so it doesn't flash.

Card chrome (`src/components/home/OnboardingChecklist.tsx`:74-77), with a green done/total progress ring (:62): - `homeChecklistTitle`: 'בוא נתחיל' rendered as ``${he.homeChecklistTitle} <`` — `src/i18n/he.ts`:1990 - `homeChecklistSubtitle`: 'כמה צעדים קטנים כדי להפיק את המקסימום' — :1991

The five steps (`ProfileScreen.tsx`:635-710), in render order, all **undone** for a new user:

#	label	he.ts	done condition	tap →
1	<code>homeStepPhoto</code> : 'הוספת תמונת פרופיל'	:199 2	<code>!!user.photoUrl</code>	<code>ProfileEdit</code> (:646)
2	<code>homeStepAvailability</code> : 'סמן מתי אתה פנוי'	:199 3	<code>preferredDays.length > 0</code>	<code>AvailabilityEdit</code> (:660)
3	<code>homeStepCommunity</code> : 'הצטרף או פתח מועדון'	:199 4	<code>myCommunities.length > 0</code>	<code>CommunitiesTab</code> (:673)
4	<code>homeStepGame</code> : 'הצטרף או צור מחזור ראשון'	:199 5	<code>totalGames > 0 myGames.length > 0</code>	<code>GameTab</code> (:687)
5	<code>homeStepInvite</code> : 'הבא חבר למגרש'	:199 6	<code>(referralCount ?? 0) > 0</code> — i.e. someone actually signed up through the link, not merely that share was tapped (:701-703)	fires the share sheet (:709)

A `done` row is rendered `disabled` (`src/components/home/OnboardingChecklist.tsx`:85) with a green checkmark-circle ; an `undone` row shows an empty checkbox (:106-110).

Note the **ordering conflict**: the checklist is positioned *below* the hero and the action tiles by deliberate choice (comment at `ProfileScreen.tsx`:1367-1369 : "ALWAYS shown while incomplete ... but positioned low"). For a user with an empty hero slot (§1.5), the single most onboarding-relevant card on the screen sits fifth.

1.10 ⑧ ROTATING "ידעת..." TIPS — ALWAYS RENDERED

`<DidYouKnowCard tips={homeTips} />` — `ProfileScreen.tsx`:1376 . Component

`src/components/home/DidYouKnowCard.tsx`:19-60 ; auto-advances every `ROTATE_MS = 6000` (:17 , :24-30) with a dot indicator (:48-58).

Header: `homeDidYouKnowTitle: '...ידעת ש' — src/i18n/he.ts:2004` . The five tips

(`ProfileScreen.tsx:723–729`): 1. `homeTipAutoTeams: 'אפשר ליצור כוחות מאוזנים אוטומטית לפי דירוג' — src/i18n/he.ts:2005` → `CommunitiesTab` 2. `homeTipInternalRating: 'דירוג פנימי של שחקנים עוזר לאזן' — src/i18n/he.ts:2006` → `CommunitiesTab` 3. `homeTipAvailability: 'סמן מתי אתה פנוי – ומנהלים' — src/i18n/he.ts:2007` → `AvailabilityEdit` 4. `homeTipScheduled: 'אפשר לתזמן מראש מתי' — src/i18n/he.ts:2008` → `GameTab/GameCreate` 5. `homeTipCommunity: 'פתח מועדון כדי לנהל' — src/i18n/he.ts:2009` → `CommunitiesTab`

All five describe **admin** features. None of them is actionable by a user who is not in a club.

1.11 ⑨ INVITE-FRIENDS CTA – ALWAYS RENDERED, LAST BLOCK

`ProfileScreen.tsx:1379–1393` . `profileInviteFriendsCta: 'הזמן חברים לאפליקציה' — src/i18n/he.ts:3020` . Handler `handleShareInvite` (`ProfileScreen.tsx:948–971`) builds a short link via `createShortInviteUrl({ type: 'app', invitedBy: user.id, ... })` and shares: - `profileInviteShareBody: (link) => \ תוריד גם אתה ובואת\ Teamder 🌐 כדורגל בעזרת אפליקציית \ — src/i18n/he.ts:3021–3022`

1.12 THE ≡ HAMBURGER – THE WHOLE REST OF THE APP

`<HamburgerMenu visible={menuOpen} sections={sections} /> — ProfileScreen.tsx:1398–1401` ; sections built at `ProfileScreen.tsx:975–1104` . Nothing here is on the home body. Sections and items: - `profileMenuSectionProfile` — `profileSectionMyAchievements` , `statsMenuLabel` , `profileEdit` , `friendsTitle` (behind `rcBool('feature_friends')` , :1013), `referralsScreenTitle` (behind `rcBool('feature_referrals')` , :1023) - `profileMenuSectionGames` — `profileSectionAvailability` , `profileSectionHistory` - admin-only approvals section, only if `(isAdmin && pendingApprovals > 0)` (:1050–1068) - `profileMenuSectionSystem` — notifications, blocked users - `profileMenuSectionSupport` — report bug / suggest feature (behind `rcBool('feature_feedback')`), rate app - `profileMenuSectionAccount` — sign out, delete account

1.13 THE GUEST BRANCH OF THE SAME SCREEN

if (localUser?.isGuest) { ... } returns a completely different screen and **none of the above renders** — `ProfileScreen.tsx:1109–1133` : - `guestProfileTitle: 'הפרופיל שלך מחכה' — src/i18n/he.ts:1809` - `guestProfileBody: 'אתה גולש כאורח. הירשם כדי לשמור מחזורים, להצטרף למועדונים ולבנות פרופיל' — src/i18n/he.ts:1810–1811` - Button `guestRegisterCta: 'הרשמה' — src/i18n/he.ts:1808` → calls `signOut()` (`ProfileScreen.tsx:1126`), which drops the guest back to the auth stack.

2. Every path onward from home, tap by tap

Six exits exist from the empty home screen. There are no others.

A. Coach CTA → **discover clubs**. Tap `'גלה מועדונים' (ProfileScreen.tsx:1177` → `handleAssistantCta` → `case 'discoverClubs': nav.navigate('CommunitiesTab')` , :924–926). **1 tap**.

B. Tile "פתח מחזור" → **create-game chooser**. `nav.navigate('GameTab', { screen: 'GamesList', params: { openCreate: true } })` (`ProfileScreen.tsx:1303–1306`). `GamesListScreen` consumes `openCreate` in an effect and calls `handleCreate()` (`src/screens/games/GamesListScreen.tsx:325–334`), which opens the chooser modal. **1 tap to the chooser**.

C. Tile "סמן זמינות" → AvailabilityEdit (ProfileScreen.tsx:1313). Also reachable from the availability prompt card (:1358) and checklist step 2 (:660).

D. Tile "הצטרף למחזור" → GameTab (ProfileScreen.tsx:1321), landing on GamesList .

E. Checklist step 3 "הצטרף או פתח מועדון" → CommunitiesTab (ProfileScreen.tsx:673).

F Bell → Requests (ProfileScreen.tsx:1164). See §6 for why this is a dead end for a new user.

2.1 PATH: CREATE A CLUB

CommunitiesTab → PublicGroupsFeedScreen . Create is reachable three ways, all calling handleCreate(source) → nav.navigate('CommunitiesCreate') (src/screens/communities/PublicGroupsFeedScreen.tsx:428-433): 1. the floating + FAB, bottom-LEFT under RTL (:866-884), label communitiesCreateGroup: 'צור מועדון חדש' (src/i18n/he.ts:1276); 2. the 'המועדונים שלי' empty-hint card's button (:810-818); 3. the whole-screen empty state's button (:761-769). Buttons 2 and 3 read communitiesCreateFirst: 'צור מועדון ראשון' — src/i18n/he.ts:1280 .

CreateGroupScreen (src/screens/groups/CreateGroupScreen.tsx) is a thin shell over GroupWizardForm , header createGroupTitle: 'יצירת מועדון חדש' (src/i18n/he.ts:1296), submit createGroupSubmit: 'צור' (:1401).

Two steps (src/screens/groups/GroupWizardForm.tsx:144 , indicator at :263-267): - Step 1 groupFormTabDetails: 'פרטי המועדון' (src/i18n/he.ts:542) - section groupSectionIdentity: 'פרטי המועדון' (:955): groupCreateName: 'שם המועדון' (:2866 , required, placeholder 'חמישי כדורגל' hardcoded at GroupWizardForm.tsx:286); createGroupDescription: 'תיאור המועדון (לא חובה)' (:1304); communityDetailsRules: 'לדוגמה: n\ח- מגיעים בזמן n\ח- ' (:1461 , placeholder hardcoded 'חוקי המועדון') - section groupSectionLocation: 'מיקום ויצירת קשר' (:1391 , hint 'להקליד שם עיר' (:1298); createGroupCity: 'עיר' (:1297 , required, placeholder createGroupCityPlaceholder: 'התחל' (:1391 , hint 'יוצג כפתור "פתח ב-WhatsApp" במועדון' (:1392); createGroupContactPhone: 'טלפון איש קשר' (:1390 , placeholder '050-1234567' (:1391 , hint 'יוצג כפתור "פתח ב-WhatsApp" במועדון' (:1392); gate: step1Valid = name and city non-empty (GroupWizardForm.tsx:212); when blocked the disabled 'המשך' is explained by gameWizardMissingFields: (fields) => \יש למלא: \${fields}\` (src/i18n/he.ts:857 , rendered GroupWizardForm.tsx:432-436) - Step 2 groupFormTabAdvanced: 'מתקדם' (src/i18n/he.ts:543) - groupSectionAccess: '957: (הצטרפות והרשאות) with the single most consequential switch in the whole funnel: createGroupIsOpen: '1307: (מועדון פתוח) , hint createGroupIsOpenHint: 'שחקנים חדשים מצטרפים' (:1308); Default is false - EMPTY_GROUP_FORM_VALUES feeds the create screen (CreateGroupScreen.tsx:140) and submitJoinByPublic reads !!pub.isOpen (src/services/groupService.ts:2030). - groupSectionRating: 'דירוג: 958: (וכרטיסים) - internal rating, hide-rating, cards toggles. - Footer buttons: wizardStepBack: '1123: (חזרה) , wizardStepNext: '1124: (המשך)'.

On success it does nav.replace('CommunityDetails', { groupId, celebrate: true }) (CreateGroupScreen.tsx:92-95).

Tap count: + (1) → type name → type city → המשך (2) → צור והיכנס (3). Three taps plus two text fields. Creating a club does **not** create a game and does **not** register the user for anything.

2.2 PATH: JOIN A CLUB

PublicGroupsFeedScreen (src/screens/communities/PublicGroupsFeedScreen.tsx). Layout per its own header comment (:1-12): hero, search+filter row, מועדונים פתוחים , מועדונים לאישור , המועדונים שלי , FAB.

Hero (src/components/community/CommunitiesHero.tsx:63-66): communitiesTitle: 'מועדונים'
(src/i18n/he.ts:1274) + communitiesHeroSubtitle: 'כל המועדונים במקום אחד' (:1408). Search
placeholder: communitiesCardSearchPlaceholder: 'חיפוש מועדון או עיר' (:1414 , used :556). Controls
row also carries a map button (mapButtonLabel: 'תצוגת מפה' , :73), a filter button (gameFiltersButton:
'סינון' , :103) and a RequestsBell (:752).

Whole-screen empty state when totalKnown === 0 && !isSearching (:753-770 ; totalKnown = (items ??
[]).length , :537) — i.e. the public club directory is genuinely empty: - communitiesEmptyAll: 'אין עדיין'
מועדונים' — src/i18n/he.ts:1278 - communitiesEmptyAllSub: 'תהיה הראשון להקים מועדון כדורגל באזור'
'שלך' — :1279 - button communitiesCreateFirst: 'צור מועדון ראשון' — :1280 - the FAB is hidden in this
state (:870)

Normal state. Section communitiesSectionMember: 'המועדונים שלי' (:1405) is **always rendered**, and for a
new user shows an empty-hint card (:795-820): - communitiesEmptyMember: 'עדיין לא הצטרפת לאף מועדון'
— :1416 - communitiesEmptyMemberSub: 'הצטרף למועדון מהרשימה למטה כדי לראות מחזוריים, או פתח'
'מועדון משלך' — :1417-1418

Section communitiesSectionPending: 'ממתנים לאישור' (:1406) renders only when pendingItems.length >
0 (:823).

Section communitiesSectionOpen: 'מועדונים פתוחים' (:1407) is the discovery list. Empty →
communitiesEmptyOpenSection: 'אין מועדונים פתוחים נוספים' (:1419). Windowed to DISCOVERY_PAGE =
8 (:74) with communitiesShowMore: (n) => \ הצג עוד מועדונים ({n}) ' (:1422 , rendered :844).

Search-mode: no-match copy is communitiesEmpty: 'לא מצאנו מועדון בשם זה. נסה לחפש לפי עיר או שם'
אחר.' (:1277 , rendered :788).

The discovery query is groupService.listPublicGroups() — an *unfiltered* getDocs(col.groupsPublic())
with no where at all (src/services/groupService.ts:634-647). Filtering, windowing and sorting are all client-
side: - exclude clubs with an empty name (comment: partial /groupsPublic docs from hidden personal groups) —
PublicGroupsFeedScreen.tsx:304-308 - exclude member / admin / pending — :309-311 - apply the user's
filters (passesDiscoveryFilters) — :312 - sort: gamesLast30 desc, then gamesLast60 desc, then
memberCount desc, then Hebrew name — :329-335 . The long comment at :317-328 explains this was changed
away from member-count sorting because *"of 53 real clubs only 15 have ever opened a round, and the ones that never
did are not the small ones — the biggest club in the app has 40 members and no fixture."*

The card is src/components/community/ClubCard.tsx , its state machine is the pure resolveClubCard
(src/utils/clubCard.ts:108-152). CTA resolution (src/utils/clubCard.ts:122-128):

relation	CTA value	label	he.ts
admin or member	member	clubCardMember: 'חבר במועדון'	:3351
pending	requested	clubCardRequested: 'בקשה נשלחה'	:3354
none + isOpen	join	clubCardJoin: 'הצטרף'	:3352
none + closed	request	clubCardRequest: 'בקש להצטרף'	:3353

Only join and request are tappable (ctxActs , ClubCard.tsx:95 , :154). Badges: clubCardManager:
'מנהל' (:3355), clubCardRecommended: 'מתאים לך' (:3356 , = within 10 km and ≥10 members,
src/utils/clubCard.ts:64-65 , :133-136), and activity clubCardVeryActive: 'פעיל מאוד' /

clubCardActive: 'פעיל' / clubCardInactive: 'לא פעיל' (:3357–3359) from clubActivity thresholds VERY_ACTIVE_MIN = 4, ACTIVE_MIN = 2 over 30 days (src/utils/clubCard.ts:83–98). A club whose counters are null gets no badge (:93). Member count line: clubCardPlayers: (n) => \ \${n} 3360: (שחקנים').

Tapping the CTA runs handleRequest (PublicGroupsFeedScreen.tsx:345–424): - guest gate first: ensureNotGuest(he.guestRegisterJoinCommunity, { type: 'team', id: item.id }) (:347) — guestRegisterJoinCommunity: 'כדי להצטרף למועדון צריך חשבון. רוצה להירשם עכשיו?' (:1804) - requestJoinById → submitJoinByPublic (src/services/groupService.ts:2024–2074): reads /groupsPublic/{id}, branches on isOpen, enforces a capacity gate on memberCount >= maxMembers throwing GROUP_FULL (:2033–2043), then writeJoin(groupId, userId, isOpen) (:2047) and returns status: isOpen ? 'joined' : 'pending' (:2072) - toasts (PublicGroupsFeedScreen.tsx:376–423): - pending → toastJoinRequestSent: 'הבקשה נשלחה' (:228) - joined → toastJoinedGroup: 'ברוך הבא למועדון' (:229) - already member → groupAlreadyMember: 'אתה כבר במועדון הזה' (:2878) - rejected → toastJoinRejected: 'בקשתך למועדון זה נדחתה ולא ניתן לשלוח שוב' (:242) - GROUP_FULL → toastGroupFull: 'המועדון מלא. לא ניתן לשלוח בקשה כרגע' (:241) - anything else → toastRequestFailed: 'שליחת הבקשה נכשלה. נסה שוב' (:237)

Tapping the card body (not the CTA) for a non-member routes to CommunityDetailsPublic — the read-only preview, src/screens/communities/CommunityDetailsPublicScreen.tsx. It shows description + communityDetailsAbout: 'תיאור המועדון' (:1452), communityDetailsCity: 'עיר' (:1454), communityDetailsField: 'מגרש' (:1453), communityDetailsPreferredDays: 'ימי מחזור' (:1455), communityDetailsPreferredHour: 'שעת מחזור' (:1456), communityDetailsMembers: 'שחקנים' (:1468). Optional WhatsApp button communityDetailsContactAdmin: 'צור קשר עם המנהל' (:1544, shown only when phoneValid, :341). Primary button copy is group.isOpen ? he.communityJoinAuto : he.communityRequestToJoin (:295): - communityJoinAuto: 'הצטרף למועדון' — :1445 - communityRequestToJoin: 'בקש להצטרף' — :1446 Once pending the button becomes groupsActionPending: 'בטל בקשת' and is disabled (:361–366), with a secondary communityCancelJoinRequest: 'הצטרפות' (:1447, rendered :371) whose success toast is toastJoinRequestCancelled: 'בקשת ההצטרפות בוטלה' (:1448).

The public preview shows the club's upcoming games nowhere. Nothing in CommunityDetailsPublicScreen.tsx renders a game list — a prospective member cannot see whether the club has a fixture this week before committing to a join request.

Tap count (open club): CommunitiesTab (1) → הצטרף on a card (2) = **2 taps, instant membership.** **Tap count (closed club, the default):** same 2 taps, then an unbounded wait for an admin.

2.3 PATH: FIND AN OPEN GAME

GameTab → GamesListScreen (src/screens/games/GamesListScreen.tsx). Full breakdown in §3.

2.4 PATH: ACCEPT AN INVITE

Two link shapes, both handled by the single consumer in RootNavigator (src/navigation/RootNavigator.tsx:62–166). Readiness gate: if (!currentUser || !profileComplete || !hasCompletedOnboarding) return (:65) — an invite therefore does **not** fire until the user has finished signup and post-signin onboarding, and consumedRef fires it at most once per launch (:62, :66).

- pending.type === 'app' — a generic app invite. It **clears the stash and does nothing else** (:77–80):
"Attribution already landed at signup ... nothing to navigate to, so just clear the stash and leave the user on home." A user who followed a friend's profileInviteShareBody link lands on the empty home screen with no

acknowledgement.

- `pending.type === 'session' → pre-flight gameService.getGameById ( :98 ) → navigateInvite(...) → MatchDetails.`
- `pending.type === 'team' → pre-flight groupService.getPublic ( :100 ) → CommunityDetails (member) or CommunityDetailsPublic (non-member), chosen by cachedGroups.some(...) ( :134–138 ).`
- Deleted target → `toast.error('הקישור לא תקין או שהפריט כבר לא קיים')` — **hardcoded Hebrew, not in he.ts**, `src/navigation/RootNavigator.tsx:124`.
- `ACCESS_BLOCKED` is treated as `exists = true` so the target screen can render its own blocked UI (:107–110).

The same stash is reused as the post-signup return target by the guest gate (`src/utils/guestGate.ts:39–63`): choosing `'הרשמה'` stashes the target and calls `signOut()` (:51–58).

3. The open-games feed: what a stranger sees

3.1 THE QUERIES

`GamesListScreen.reload()` runs four in parallel (`src/screens/games/GamesListScreen.tsx:246–254`):

```
gameService.getMyLiveOrUpcomingGames(user.id)
gameService.getCommunityGames(user.id, myCommunityIds)
gameService.getOpenGames(user.id, myCommunityIds)
gameService.getMyUpcomingScheduledGames(user.id, myCommunityIds)
```

`getOpenGames` is the only one that can return anything for a user with no club

(`src/services/gameService.ts:2517–2570`). The server-side query, verbatim (:2547–2554):

```
query(
  col.games(),
  where('visibility', '=', 'public'),
  where('status',    '=', 'open'),
  where('startsAt',  '>', now),
)
```

Then client-side (:2561–2570): re-check `status === 'open'`, `startsAt > now`, `!isStaleAfterStart(g)`, `!excludeCommunityIds.includes(g.groupId)`, and drop games where the user is already in `players / waitlist / pending`. Sorted `startsAt` ascending.

So the answer to "what determines which games a stranger sees" is: `visibility === 'public' AND status === 'open' AND startsAt > now`, and nothing else.

- **No city filter.** None.
- **No distance filter.** `EMPTY_GAME_FILTERS` has `nearby: false` (`src/utils/gameFilters.ts:25–33`); `DEFAULT_GAME_NEARBY_RADIUS_KM = 25` (:23) only applies once the user opts in through the filter sheet. Default feed shows a game in Eilat to a user in Haifa.

- `acceptsFillers` **plays no part in the feed**. `grep -rn "acceptsFillers" src` returns only the type (`src/types/index.ts:1549` , `:2028`), the create/edit wizard (`GameCreateScreen.tsx:134` , `:503-504` ; `GameEditScreen.tsx:110` , `:340-341` ; `GameWizardForm.tsx:195` , `:774`), the series scheduler (`src/utils/seriesSchedule.ts:201` , `:284`) and the admin review section (`src/components/match/FillerInterestsSection.tsx:10`). It is **not** in any feed query. Its only client-side consumer is the filler-candidate banner on `MatchDetails` (`src/screens/games/MatchDetailsScreen.tsx:810`) — i.e. it changes what you can do once you are *on* a game, not whether you can *see* it.
- `getCommunityGames` (`src/services/gameService.ts:2239-2313`) is `where('groupId','in',chunk) + where('status','==','open')` , chunked at 30, fail-soft per chunk, and drops games the user is already in (`:2304-2309`). `communityIds.length === 0 → return []` (`:2243`). Useless for a new user.
- `getMyUpcomingScheduledGames` (`:2328-2389`) is `where('groupId','in',chunk) + where('status','==','scheduled')` , same empty-club guard (`:2332`), sorted by `registrationOpensAt ?? startsAt` (`:2338`).

Render-layer visibility: `isVisibleInOpenGames(g) = isOpen(g) && (!hasStarted(g) || canJoinGame(g))` (`src/services/gameLifecycle.ts:310-315`) — a game stays in discovery for the 1h late-registration grace after kickoff.

3.2 WHAT THE SCREEN RENDERS

`Hero MatchesHero` (`src/components/match/MatchesHero.tsx:38-81`), pinned outside the scroll: - `gamesListTitle: 'מחזורים'` (`src/i18n/he.ts:793`) — the comment at `MatchesHero.tsx:39-41` notes the greeting was deliberately moved to Home - `matchesHeroSubtitle: 'הצטרף למחזורים או צור מחזור חדש'` (`:3071`)

Floating controls row (`GamesListScreen.tsx:653-855`): map button, filter button with a count badge, requests bell with a count badge.

The list is **one sectioned scroll, no tabs** (comment `:654-655`), with three possible bodies:

(a) `filteredToNothing` — `isEmpty && filterCount > 0` (`GamesListScreen.tsx:578`) → `FullEmptyState` with `hasActiveFilters` (`:891-899` , component `:1324-1390`): - `emptyHomeFilteredBody: 'אין מחזורים לסינון'` — `:2853` - button `emptyHomeClearFilters: 'נקה סינון'` — `:2854` - the FAB is **hidden** in this state only (`:975`) - (unreachable branches of the same component: `emptyHomeBody: 'צור מחזור חדש או'` `:2844` , `emptyHomeNoGamesAnywhere: 'היה הראשון לפתוח'` `:2845` , `emptyHomePrimary: 'צור מחזור'` `:2846` , `emptyHomeSecondary: 'מצא מחזור'` `:2846` — `FullEmptyState` is now only ever called with `hasActiveFilters` , `GamesListScreen.tsx:891-899`)

(b) `isEmpty with no filters` — **THE NEW-USER CASE** (`GamesListScreen.tsx:900-912`): - `gamesNoOpenTitle: 'אבל יש אנשים'` — `src/i18n/he.ts:3371` - `gamesNoOpenBody: 'אין כרגע מחזורים פתוחים שמתאימים לך'` — `:3372` - then `upcomingSection` (empty for a new user), then `MatchEmptyHintCard` , then `discovery`

(c) **Non-empty** (`:913-965`) — `matchesSectionMine: 'המחזורים שלי'` (`:3365`), `homeUpcomingSectionTitle: 'בקרוב במועדונים שלך'` (`:2030`), `matchesSectionOpen: 'מחזורים פתוחים'` (`:3364`), then `discovery` below every real match.

`MatchEmptyHintCard` (`src/components/match/MatchEmptyHintCard.tsx:25-48`): - `matchesEmptyCardTitle: 'צור מחזור חדש ותן לאחרים להצטרף'` — `:3400` - `matchesEmptyCardSub: 'לא מצאת מחזור מתאים?'` — `:3399`

The FAB (GamesListScreen.tsx:976–999): pinned bottom-LEFT under RTL, Breathing pulse, accessibility label matchesCreateFab: 'יצירת מחזור חדש' (:139). A one-time device hint bubble points at it: hintCreateGame: 'כאן יוצרים מחזור חדש' (:2857), shown when storage.getHintCreateGameSeen() is false (:225–232).

A first-visit modal also fires here. AvailabilityNudgeModal (GamesListScreen.tsx:1101–1115) is armed when the user has no preferredDays and hasn't been shown it in 3 days (:121–132) — so a brand-new user's very first visit to the Games tab is interrupted by a popup before they see the list.

3.3 THE DISCOVERY BLOCKS BENEATH THE LIST

Shown when !filteredToNothing && density !== 'many' (GamesListScreen.tsx:582). Density from feedDensity(visibleGamesCount, richMin) (src/utils/feedDensity.ts:38–41): 0 → 'none', >= richMin → 'many', else 'few'. Defaults richMin: 5, demandMin: 3, clubsMax: 3, clubsRadiusKm: 30, clubsMinMembers: 10 (src/utils/feedDensity.ts:12–27), each overridable by Remote Config (src/config/gamesFeedDiscovery.ts:42–56).

AreaDemandCard (src/components/games/AreaDemandCard.tsx) — without a saved home area it shows the prompt variant (:114–126): - gamesDemandPromptTitle: 'רוצה לדעת מי מחפש משחק לידך?' — src/i18n/he.ts:3391 - gamesDemandPromptBody: 'הגדר את אזור הבית והימים שנוח לך, ונראה לך כמה' — 3392–3393 - gamesDemandPromptCta: 'הגדר אזור וזמינות' — 3394 With data (:150–216): gamesDemandTitle: 'ביקוש' — 3383–3384: (באזור שלך), gamesDemandHeroSub: 'נראה שיש פה מחזור שמחכה' — 3385: (לקרות), gamesDemandOpenCta: (day) => 'פתח מחזור' — 3386: (day) => 'אני' — 3387–3388: gamesDemandLookingHeadline (day) => 'הספירה מבוססת על שחקנים שסימנו זמינות' (פנוי), gamesDemandFoot: 3390: (פנוי).

NearbyClubsSection (src/components/games/NearbyClubsSection.tsx:186–194): - with location: gamesClubsNearbyTitle: 'מועדונים באזור שלך' (:3395) / gamesClubsNearbySub: 'קהילות כדורגל' — 3396 - without: gamesClubsAnyTitle: 'מועדונים שאפשר' — 3397 / gamesClubsAnySub: 'קהילות הפעילות באפליקציה — הגדר אזור בית כדי לראות את' (:3398) It carries its own inline join with the identical toast set as the communities feed (NearbyClubsSection.tsx:126–173), so a club can be joined **without ever opening the Communities tab**.

3.4 THE CARD AND ITS BUTTON, STATE BY STATE

MatchListCard (src/components/match/MatchListCard.tsx). Occupancy counts registered players **plus active guests plus a held promotion reservation** (:103–106):

```
occupancy = game.players.length + activeGuestCount(game.guests) + (game.pendingPromotion?.uid ? 1 : 0)
```

ctaForGame (:63–86), in order: 1. status === 'joined' → cancel 2. status === 'waitlist' → leaveWaitlist 3. status === 'pending' → pending 4. g.status === 'scheduled' → none 5. occupancy >= g.maxPlayers → waitlist — **occupancy is checked before** requiresApproval, with a long comment at :71–78 explaining the production case: "The one public game in the country is sitting at exactly 21/21 (1 player + 20 active guests) and has collected 7 such requests over 17 days, aged up to 18." 6. g.requiresApproval → requestJoin 7. else → join

Only `join / requestJoin / waitlist` render a button (`showCta` , :135–136). The label is deliberately only two possibilities (:139–140): - `gameCardRequestJoin`: 'בקש להצטרף' — `src/i18n/he.ts:808` (approval required) - `matchCardJoinShort`: 'הצטרף' — :3033 (everything else, **including a full game** — comment at `MatchListCard.tsx:137–138`: "Even a full game's CTA just says 'הצטרף' ... tappers land on the waitlist and then see the 'ברשימת המתנה' badge.")

Otherwise a non-interactive status badge replaces the button (:145–152): - `joined` → `matchStatusJoined`: 'בהרכב' (:3048), green - `waitlist` → `matchCardInWaitlist`: 'ברשימת המתנה' (:3034), amber - `pending` → `matchStatusPending`: 'ממתין לאישור' (:3050), grey

Other card copy: occupancy `matchCardOccupancy`: (n, max) => `\ ${n}/${max}` (:3041); `when-line` `matchCardWhenToday`: (t) => `היום-${t}` (:3037), `matchCardWhenTomorrow`: (t) => `מחר ב-${t}` (:3038), `matchCardWhenDate`: (d, t) => `-${d} · ${t}` (:3039); `visibility` `chip` `matchTagOpenToAll`: '3052: (פתוח לכולם) / `matchTagCommunityOnly`: '3053: (סגור למעדון) / `matchTagQuickClosed`: '3054: (מחזור מהיר' (`MatchListCard.tsx:159–167`).

Progress-bar urgency: red when full, amber at ≤ 3 spots, green otherwise (:113). Side stripe: green if joined, amber if waitlisted, blue otherwise (:128–129).

3.5 JOINING FROM THE CARD

`handleCardPrimary` (`GamesListScreen.tsx:337–473`): - `cancel` past the deadline → `ConfirmDestructiveModal` with `lateCancelTitle`: 'ביטול קרוב מאוד למחזור' (:251) / `lateCancelConfirm`: 'אישור ביטול' (:254) - `cancel` otherwise → `appAlert`(`he.leaveGameConfirmTitle`, `he.leaveGameConfirmBody`, ...) (:352–362) — `leaveGameConfirmTitle`: 'לבטל את ההרשמה?' (:3802), `leaveGameConfirmBody`: 'ההרשמה שלך למחזור תבוטל. תמיד אפשר להירשם שוב כל עוד יש מקום.' (:3803), destructive label `matchMenuLeave`: 'יציאה מהמחזור' (:3801) - `join | requestJoin | waitlist` → `gameService.requestJoinGame`(`game.id`, `user.id`, 'games_list') and then a toast keyed on the **returned bucket** (:372–380): - `toastGameJoinedWaitlist`: 'נוספת לרשימת המתנה' — :234 - `toastGameJoinedPending`: 'בקשת ההצטרפות נשלחה' — :235 - `toastGameJoined`: 'הצטרפת למחזור' — :231 - the card is then patched optimistically in place and reconciled after 2500 ms (:381–419), because the fair-queue seating is an async Cloud Function - error branches (:420–470): `REGISTRATION_CONFLICT` → the conflict modal, fallback toast `registrationConflictTitle`: 'אתה כבר רשום למחזור בזמן חופף' (:3816); `GAME_JOIN_REJECTED` → `matchDetailsJoinRejected`: 'בקשתך למחזור זה נדחתה ולא ניתן להירשם שוב.' (:3422); `GAME_NOT_OPEN | GAME_STARTED | GAME_LIVE` → `gameNotJoinableToast`: 'המחזור כבר לא פתוח להרשמה' (:3815)

There is **no guest gate on the card join** — `ensureNotGuest` appears in `handleCreate` (:315) and `openCreateForSlot` (:591) but not in `handleCardPrimary` . A guest tapping `הצטרף` on a card goes straight into `requestJoinGame` .

4. Joining a game: every gate, with the button label in each state

4.1 THE LIFECYCLE GATE (SHARED CONTRACT)

`src/services/gameLifecycle.ts` is the single source, explicitly framed as "the contract" (:1–5).

`canJoinGame`(`game`) (:150–157) returns false unless all three hold: 1. `isOpen`(`game`) — `effectiveStatus` === 'open' ; `status` === 'open' && `liveMatch.phase` === 'live' is normalised to 'active' (:43–48 , :65–67) 2. `!isPastLateRegistrationCutoff`(`game`) — `LATE_REG_GRACE_MS` = `60 * 60 * 1000` , i.e. joining stays open 1

hour past kickoff (:120 , :135–138) 3. **!isRoundRunning(game)** — phases 'live' and 'roundRunning' (:50–58)

Capacity is explicitly **not** considered (comment :147–148): *"a full game still allows joining the waitlist."*

canCancelRegistration(game) = **isOpen(game) || isLocked(game)** (:167–169). **isStaleAfterStart** = **kickoff + 6 h** (:121 , :129–132).

4.2 THE WRITE PATH

requestJoinGame (**src/services/gameService.ts:5136–5285**), fair-queue registration: - **tappedAt** = **serverNow()** captured **before any await** (:5141) so latency can't reorder the queue - Firestore pre-checks (:5233–5265), in order, each a thrown code: - doc missing → 'requestJoinGame: game not found' - **data.status !== 'open'** → **GAME_NOT_OPEN** (:5236) - **data.startsAt + LATE_REG_GRACE_MS < Date.now()** → **GAME_STARTED** (:5240) - **data.liveMatch?.phase === 'live'** → **GAME_LIVE** (:5242) - **(data.rejectedPlayerIds ?? []).includes(userId)** → **GAME_JOIN_REJECTED** (:5243–5247) - already in **players / waitlist / pending** → returns that bucket, idempotent (:5248–5258) - **findRegistrationConflict** → **REGISTRATION_CONFLICT** with the clashing game attached (:5259–5265) - then a contention-free per-user write to **games/{id}/joinRequests/{uid}** with **state: 'queued'** (:5276–5281), preceded by a **deleteDoc** so a re-join is always a fresh CREATE (rules forbid updating the doc — comment :5268–5274) - returns an **optimistic** bucket from **predictBucket / assignJoins** (:5145–5170 , :5284); the server reconciler is authoritative

Bucket prediction inputs (:5159–5168) include the key carve-out: **requiresApproval: g.requiresApproval === true && g.createdBy !== userId** — **the creator never has to approve themselves** — and **pendingOfferReservation: !!g.pendingPromotion?.uid**.

4.3 THE MATCHDETAILS SCREEN'S BUTTON, EVERY STATE

src/screens/games/MatchDetailsScreen.tsx.

statusForUser (:154–159): **joined** → **waitlist** → **pending** → **none**. **wasRejected** = **(game.rejectedPlayerIds ?? []).includes(user.id)** (:1375–1376). **isFull** = **totalParticipants + (pendingPromotion ? 1 : 0) >= maxPlayers** (:1394–1395) — includes guests (:1378–1379). **needsApproval** = **game.requiresApproval === true && !isAdmin && !isInvitedToGame** (:1445), where **isInvitedToGame** = **(game.invitedUserIds ?? []).includes(user.id)** (:1443–1444) — comment at :1439–1442 : *"the creator + an invited player both saw 'בקש להצטרף' on a game they were invited to."*

primaryLabel (:1446–1451), in order:

condition	label	he.ts
primaryDestructive (joined / waitlist / pending)	matchDetailsCancel: 'בטל הרשמה'	:3414
isFull && !needsApproval	gameStatusWaitlist: 'הצטרף לרשימת המתנה'	:821
needsApproval	gameCardRequestJoin: 'בקש להצטרף'	:808
else	matchDetailsJoin: 'הצטרף למחזור'	:3415

But **primaryLabel** is not always what renders. The **primary** builder (:1912–2022) can return **null** or override it: 1. **isTerminalGame(game)** → **null** (:1913) 2. **game.pendingPromotion?.uid === user.id** → **matchDetailsAcceptOffer:** 'נפתח לך מקום – אשר הגעה' (:3416), wins over everything (:1916–1922) 3. **admin branches** (:1923–1981): **sessionActionInvitePlayers:** 'הזמן שחקנים' (:292) or **sessionActionGoLive:** 'עבור ללייב' (:295) 4. registered participant + game active → **sessionActionGoLive**

(:1994–2005) 5. if (primaryDestructive) return null (:2008) — a registered, waitlisted or pending user gets NO button at all on this screen 6. if (wasRejected) return null (:2011) — rejected users see no join CTA 7. if (isFillerCandidate) return null (:2016) — the filler banner owns that flow instead 8. else { title: primaryLabel, onPress: handlePrimary } (:2017–2021)

blockedByConflict = !!preCheckConflict && !!primary && status === 'none' (:2025–2026). When true the sticky bar shows a locked, shake-on-tap button (:2612–2622) labelled matchPrimaryConflict: 'יש לך מחזור' (:3784).

Cancelling is menu-only. Comment at :2641–2643 : "the giant red 'בטל הרשמה' is gone; the only exit is the ≡ menu's 'יציאה מהמשחק'", gated on primaryDestructive && canCancelRegistration(game) (:2245), item label matchMenuLeave: 'יציאה מהמחזור' (:3801), and the confirm lives inside handlePrimary so every entry point gets it (:1414–1434).

4.4 PRE-TAP TOASTS WHEN CANJOININGAME IS FALSE

performPrimary (:901–1000), branch order at :917–932 :

condition	toast	he.ts
isFinished(game)	matchDetailsAlreadyFinished: 'המחזור הסתיים'	:3423
isCancelled(game)	matchDetailsAlreadyCancelled: 'המחזור בוטל'	:3424
isRoundRunning(game)	matchDetailsAlreadyLive: 'המחזור כבר במצב לייב'	:3421
startsAt < now	matchDetailsAlreadyStarted: 'המחזור כבר התחיל'	:3420
isScheduled(game) + registrationOpensAt	matchDetailsRegistrationOpensAt: (when) => \	
isScheduled(game) without it	communityNextGameLocked: 'ההרשמה תיפתח בקרוב'	:1589
anything else	matchDetailsClosedForRegistration: 'ההרשמה נסגרה'	:3417

Then the red-card gate (:936–965): redBlocked is re-verified against the server at tap time because the mount-time flag doesn't live-update, and on failure shows redCardBlockToast: 'יש לך כרטיס אדום פעיל במועדון – לא' (:550) as an error toast, not info.

Guest gate first of all (:906–908): ensureNotGuest(he.guestRegisterJoinGame, { type: 'session', id: game.id }) — guestRegisterJoinGame: 'כדי להירשם למחזור צריך חשבון. רוצה להירשם עכשיו' (:1803). The {type: 'session'} stash is what brings the user back to this game after signup.

4.5 WHOLE-SCREEN BLOCKED STATES

- **Not found** (:1251–1280): matchDetailsDeletedTitle / matchDetailsDeletedBody + deletedTargetBackToMain , resets to GameTab/GamesList .
- **Access blocked** — a non-member opened a community-only game and the rules denied the read (:1282–1308). Deliberately leaks nothing: header is the generic screen title, no group name, no fields.
- communityOnlyGameTitle: 'מחזור לסגל בלבד' — :3474

- `communityOnlyGameSubtitle`: 'המחזור הזה פתוח רק לחברי המועדון' — :3475
- `communityOnlyGameBack`: 'חזור' — :3476
- **This is a hard dead end.** There is no "request to join the club" affordance on this screen.
- **Load error** (:1310–1340): `matchDetailsLoadErrorTitle` / `matchDetailsLoadErrorBody` + `gameRetry`.

4.6 REGISTRATION WINDOWS

Three independent timestamps on the `Game` doc:

- `registrationOpensAt` (`src/types/index.ts:1782`) — until it passes the game's `status` is 'scheduled', set at create time: `const isDeferred = typeof input.registrationOpensAt === 'number' && input.registrationOpensAt > now; const initialStatus = isDeferred ? 'scheduled' : 'open'` (`src/services/gameService.ts:2776–2779`). A Cloud Function (`flipScheduledGames`) flips it to 'open'. While scheduled: the card CTA is `none` (`MatchListCard.tsx:70`), `getOpenGames` and `getCommunityGames` both filter it out (they require `status == 'open'`), and it appears only in the 'בקרוב במועדונים שלך' teaser section for **members of that club** (`getMyUpcomingScheduledGames`, `src/services/gameService.ts:2328–2389`). A stranger cannot see a scheduled game at all.
- `publicOpenAt` (`src/types/index.ts:1830`) — ms-epoch when a CF flips `visibility` from 'community' to 'public'. Latch `publicOpenedAt` (:1832). Until it fires, non-members simply do not match the `getOpenGames` query. Set in the wizard at `src/screens/games/GameWizardForm.tsx:1109–1130`, summarised as the hardcoded Hebrew 'המשחק ייפתח לכל האפליקציה' (`GameWizardForm.tsx:1195` — **string not in he.ts**). Toggle label `wizardPublicOpenToggle`: 'פתיחה לכלל האפליקציה בזמן מתוזמן' (`src/i18n/he.ts:1165`).
- `guestsOpenAt` (`src/types/index.ts:1840`) — gates only *adding guests*, never joining. `canAddGuest` (`src/services/gameLifecycle.ts:184–201`): open-only, admins always allowed, otherwise the actor must be a participant (`players ∪ waitlist`) and `Date.now() >= (game.guestsOpenAt ?? 0)`.

Rejected lists. `rejectedPlayerIds` (`src/types/index.ts:1626`, doc-comment :1620–1625: "Terminal: only an admin clears it.") blocks both the client CTA (`MatchDetailsScreen.tsx:2011`) and the write (`gameService.ts:5243–5247`, and :5181–5185 in mock).

Waitlist / promotion offers. `waitlistApprovalRequired` + `waitlistApprovalTimeoutMinutes` (`src/types/index.ts:1543–1544`). `cancelGameV2` sets `pendingPromotion` rather than auto-seating (comment `MatchDetailsScreen.tsx:1000–1010`). The offered user's CTA becomes `matchDetailsAcceptOffer`: 'נפתח לך' (:3416), handled by `handleConfirmSpotOffer` (:1518–1530). The held reservation counts toward occupancy everywhere (`MatchListCard.tsx:106`, `MatchDetailsScreen.tsx:1394–1395`).

5. What the user sees after requesting, before an admin approves

5.1 PENDING ON A GAME (REQUIRESAPPROVAL → PENDING[])

surface	what renders	file:line
toast at the moment of the tap	<code>toastGameJoinedPending</code> : 'בקשת ההצטרפות נשלחה'	<code>src/i18n/he.ts:235</code> , <code>GamesListScreen.tsx:376</code>
the feed card's button	replaced by a grey non-interactive badge <code>matchStatusPending</code> : 'ממתיך לאישור'	:3050, <code>MatchListCard.tsx:150–152</code>

surface	what renders	file:line
MatchDetails hero badge	<code>matchStatusPending: 'ממתין לאישור', tone="neutral"</code>	<code>MatchDetailsScreen.tsx:3924–3925</code>
MatchDetails status card title	<code>matchStatusCardYouRegistered: 'אתה רשום למחזור'</code>	<code>:3523</code> , chosen at <code>MatchDetailsScreen.tsx:333–335</code> via <code>const userIsIn = status !== 'none'</code>
MatchDetails status card subtitle	<code>matchStatusCardWaitingHelper: (n) => \</code> <code>שחקנים כדי להתחיל` :3521- when under `</code> <code>3522 , MatchDetailsScreen.tsx:322-325</code>	
MatchDetails button	<code>none</code> . <code>primary</code> returns null at if (<code>primaryDestructive</code>) return null, and <code>buildStatusCardProps</code> then returns kind: <code>'none'</code> , which suppresses the whole actions row	<code>MatchDetailsScreen.tsx:2008</code> , <code>:356–362</code> ; <code>src/components/match/MatchStatusCTACard.tsx:88</code>
the only way out	<code>⇒ → matchMenuLeave: 'יציאה מהמחזור'</code> , confirmed by <code>leaveGameConfirmTitle / leaveGameConfirmBody</code>	<code>:3801–3803</code> , <code>MatchDetailsScreen.tsx:2245–2257</code>

⚠ Factual mismatch worth flagging to the consultant: a user whose request is still **pending approval** is told `'אתה רשום למחזור'` ("you are registered for the match"), because `userIsIn` is computed as `status !== 'none'` (`MatchDetailsScreen.tsx:333`) and `'pending'` satisfies it. The hero badge one card above simultaneously says `'ממתין לאישור'`. Same screen, two contradictory claims.

There is **no timeline**, **no ETA**, and **no notification-status line** anywhere in the pending state. Nothing tells the user how long approval takes or who is deciding.

5.2 PENDING ON A CLUB

- toast: `toastJoinRequestSent: 'הבקשה נשלחה'` (`:228`, `PublicGroupsFeedScreen.tsx:379`)
- the club moves into its own feed section `communitiesSectionPending: 'ממתנים לאישור'` (`:1406`, rendered only when non-empty, `PublicGroupsFeedScreen.tsx:823–830`)
- its card CTA becomes the non-tappable `clubCardRequested: 'בקשה נשלחה'` (`:3354`, resolved at `src/utils/clubCard.ts:124–125`), amber-tinted (`ClubCard.tsx:183`, `:190`)
- on the public details screen the primary button becomes `groupsActionPending: 'הבקשה נשלחה'` and is disabled, with `communityCancelJoinRequest: 'בטל בקשת הצטרפות'` beneath it (`CommunityDetailsPublicScreen.tsx:361–378`)
- a pending member sees none of the club's games. `getCommunityGames / getMyUpcomingScheduledGames` are keyed off `useGroupStore.groups` (members only), never `pendingGroups`.

5.3 PENDING AS A FILLER CANDIDATE

`isFillerCandidate` = signed in, `game.acceptsFillers === true`, `game.status === 'open'`, not a member of the game's club, not already in the roster (`MatchDetailsScreen.tsx:808–819`). Banner at `:2724–2754` : - `fillerApplyTitle: 'המחזור מחפש שחקנים להשלמה'` — `:3771` - before: `fillerApplySub: 'הגש מועמדות'` — `:3772`) + a full-width button labelled `gameCardRequestJoin: 'בקש'` (`:3775`) + `fillerApplySentSub: 'הבקשה נשלחה. מנהל המחזור יחליט אם לאשר'` (`:808`) - after: `fillerApplySentChip: 'נשלח'` (`:3776`) - toast on submit: `fillerApplySent: 'ממתין'`

(:3777) - 'לא הצלחנו לשלוח, נסו שוב' failure fillerApplyError: (:3774); 'לאישור המנהל' fillerApplyCta: 'הגש מועמדות' (:3773) exists in he.ts but the banner now renders gameCardRequestJoin instead (MatchDetailsScreen.tsx:2746) — an **orphaned string** - the sticky CTA is suppressed for this user so the two request mechanisms don't co-exist (:2012-2016) - fillerState is **local component state only** (:821). Navigating away and back re-renders 'idle' and the button says 'בקש להצטרף' again, with no record that a request is outstanding.

6. Dead ends where a new user gets stuck

D1 — The empty home has no hero and no primary action. With no club, nextGame, openToJoin and scheduledUpcoming are all empty (ProfileScreen.tsx:344-399), pickHomeHero returns 'none' (src/utils/homeHero.ts:38-39), and the hero slot renders null (ProfileScreen.tsx:1258). The card written for exactly this moment — 'אין לך מחזור קרוב' / 'מצא מחזור פתוח או פתח מחזור חדש' / 'מצא מחזור' — is unreachable (§1.5). What remains above the fold is a coach's one-liner and three equal-weight tiles.

D2 — The outgoing-requests inbox does not exist. The bell at ProfileScreen.tsx:1157-1164 and GamesListScreen.tsx:836-848 opens RequestsScreen, whose data is getInboxCount / getInboxRequests — friendsService.listIncomingRequests + myAdminGroupsWithPending + myGamesWithPending (src/services/requestsService.ts:59-69, :72-97). All three are **incoming, admin-side**. A new user's own pending club and game requests appear in **no inbox anywhere**. Empty state: requestsEmpty: 'אין בקשות' (:1668), requestsEmptyHint: 'בקשות חברות, הצטרפות למועדונים ולמחזורים יופיעו כאן' (:1669) — copy that reads as if it covers outgoing requests, and does not.

D3 — A closed club with no admin response. isOpen defaults false (§2.2). The user waits with no ETA, no reminder, no channel. The only escape is the 'בטל בקשת הצטרפות' button on the public details screen (CommunityDetailsPublicScreen.tsx:371). WhatsApp contact exists only if the admin filled contactPhone (:341).

D4 — Community-only game via an invite link. The blocked screen (MatchDetailsScreen.tsx:1282-1308) shows 'מחזור לסגל בלבד' / 'המחזור הזה פתוח רק לחברי המועדון' and one button, 'חזור'. It cannot name the club (by design, :1283-1286), so the user cannot go and request to join it.

D5 — Rejected from a game. wasRejected hides the CTA permanently (:2011), tapping anything that reaches the service yields matchDetailsJoinRejected: 'בקשתך למחזור זה נדחתה ולא ניתן להירשם שוב' (:3422), and the field is terminal — "only an admin clears it" (src/types/index.ts:1624). No appeal path.

D6 — "פתח מחזור" for a user with no club. The tile routes to the chooser (ProfileScreen.tsx:1303-1306). canCreateCommunityGame requires admin of some club (GamesListScreen.tsx:552-553), so the 'מחזור למועדון' card is rendered **locked** with a padlock and: - createGameChooseCommunityLocked: 'מחזור למועדון' — 'קבוע שלך - אבל עדיין אין לך מועדון. הקם מועדון ראשון כדי לפתוח לו מחזורים' - or, for a member who isn't an admin: createGameChooseCommunityNotAdmin: 'מחזור למועדון יכול לפתוח מחזור' — 'למועדון. בקש מהמנהל שלכם לפתוח, או הקם מועדון משלך' (GamesListScreen.tsx:1178-1207): createGameCreateCommunityCta: 'הקמת מועדון ראשון' (:1162) or createGameCreateOwnCommunityCta: 'הקמת מועדון משלי' (:1163). So it is an obstacle, not a true dead end — **provided** rcBool('feature_quick_games') **is on** (GamesListScreen.tsx:1141). If that flag is off, the chooser contains only the locked card and the escape button.

D7 — Reaching `GameCreate` directly with no club. Three separate full-screen gates

(`GameCreateScreen.tsx:362–394`):- `createGameNoCommunities`: 'צריך להצטרף, לפני שתוכל ליצור מחזור, רק מנהלי מועדון יכולים ליצור מחזורים. בקש' + `OrphanCta` - `createGameNoAdmin`: 'רק מנהלי מועדון יכולים ליצור מחזורים. בקש' + `OrphanCta` `OrphanCta` (`:1196`) + `OrphanCta` `OrphanCta` (`:1197–1198`) + `OrphanCta` `OrphanCta` (`:1239`) / `createGameOrphanCtaSub`: 'צור מחזור חד-פעמי' (`:1240`). Note the **contradiction**: 'לפני' sits directly above a button that creates a game without one.

D8 — Zero public games. If `getOpenGames` returns nothing, the feed shows 'אין כרגע מחזורים פתוחים' plus discovery blocks. Not a hard stop, but the only two exits are "create a game" and "join a club", and per the sort comment in `PublicGroupsFeedScreen.tsx:317–328`, most clubs in the directory have never opened a round.

D9 — Generic app invite lands nowhere. `pending.type === 'app'` clears the stash and returns, leaving the user on the empty home (`RootNavigator.tsx:77–80`). No "X invited you" acknowledgement.

D10 — The guest cul-de-sac. A guest browsing games hits `ensureNotGuest` on join (`MatchDetailsScreen.tsx:906`), creating a club (`GamesListScreen.tsx:315`) or joining one (`PublicGroupsFeedScreen.tsx:347`). Choosing 'הרשמה' calls `signOut()` (`src/utils/guestGate.ts:58`), which unmounts the whole navigator. The return target survives only via the stash (`:51–57`) and only for `session` / `team` types.

7. Minimum tap count, account created → registered for a game

Counting discrete screen taps, not keystrokes. "Registered" = the user's uid is in `players[]`.

BEST CASE — 6 TAPS (CREATE A QUICK GAME; THE CREATOR IS AUTO-REGISTERED)

The decisive line: `const autoSelfRegister = input.isOrphanContext === true && !isDeferred; const initialPlayers = autoSelfRegister ? [input.createdBy] : []` (`src/services/gameService.ts:2788–2790`), with the reasoning at `:2781–2787`: "Quick games ('orphan context') are created by a user who wants to PLAY them — the wizard never asks whether to self-register, because it's implicit."

#	tap	file:line
1	home tile 'פתח מחזור' → <code>GamesList</code> with <code>openCreate:true</code> → chooser opens automatically	<code>ProfileScreen.tsx:1303–1306</code> ; <code>GamesListScreen.tsx:325–334</code>
2	'מחזור מהיר' in the chooser → <code>GameCreate { quick: true }</code>	<code>GamesListScreen.tsx:1155–1157</code> ; label <code>:1700</code>
—	type a field location (<code>createGameField: 'מיקום המגרש'</code> , <code>:870</code>) — the only required field, <code>GameWizardForm.tsx:343</code>	
3	<code>wizardStepNext: 'המשך'</code> (step 1 → 2)	<code>:1124</code> ; <code>GameWizardForm.tsx:554–562</code>
4	'המשך' (step 2 → 3)	same
5	<code>createGameSubmit: 'יצירת מחזור'</code> → opens the summary sheet	<code>:961</code> ; <code>GameWizardForm.tsx:563–571</code>
6	<code>wizardSummaryConfirm: 'אישור ויצירה'</code>	<code>:1190</code> ; <code>GameWizardForm.tsx:635–648</code>

Result: `nav.replace('MatchDetails', { gameId, celebrate: true })` (`GameCreateScreen.tsx:545–548`) and the creator is already in `players[]`.

Caveats that can make it more: the quick path first provisions a hidden personal group (`groupService.ensurePersonalGroupId()`, `GameCreateScreen.tsx:192`) behind a spinner, `createGameQuickLoading: 'ממכילים מחזור מהיר' (:1242, GameCreateScreen.tsx:619–629); the whole 'מחזור מהיר' option is behind rcBool('feature_quick_games') (GamesListScreen.tsx:1141); and the app's own overrides.quick blanks the title field (GameCreateScreen.tsx:62–65, :95) so most users will type a name too.`

This is "registered" in the bookkeeping sense only — there is no opponent yet.

BEST CASE WITH ANOTHER HUMAN — 2 TAPS (JOIN AN EXISTING OPEN PUBLIC GAME)

#	tap	file:line
1	home tile 'הצטרף למחזור' → <code>GameTab</code> → <code>GamesList</code>	<code>ProfileScreen.tsx:1321</code>
2	<code>matchCardJoinShort: 'הצטרף'</code> on a card in 'מחזורים פתוחים'	<code>:3033</code> ; <code>MatchListCard.tsx:139–140</code> , handler <code>GamesListScreen.tsx:370–380</code>

Requires all of: a public, open, future game exists (`gameService.ts:2547–2554`); it isn't full; `requiresApproval === false`; no schedule conflict; the user isn't in `rejectedPlayerIds`. Success toast 'הצטרפת למחזור' (`:231`). Going via `MatchDetails` instead makes it 3 (card body → 'הצטרף למחזור', `:3415`).

REALISTIC CASE — 4 TAPS OF USER ACTION PLUS AN UNBOUNDED HUMAN WAIT

For a brand-new user the public feed is usually empty (§6 D8), so the real route is through a club:

#	tap	file:line
1	'גלה מועדונים' on the coach card (or the 'מועדונים' tab) → <code>CommunitiesFeed</code>	<code>ProfileScreen.tsx:924–926</code> ; label <code>:3293</code>
2	<code>clubCardRequest: 'בקש להצטרף'</code> on a club card (default, because <code>isOpen</code> defaults false)	<code>:3353</code> ; <code>src/utils/clubCard.ts:126–128</code>
wait	an admin approves. No inbox, no ETA, no reminder for the requester (§6 D2/D3).	
3	open the club's game — from home's hero if one is open (<code>UpcomingScheduledGameCard</code> / <code>HomeNextGameCard</code> , <code>ProfileScreen.tsx:1228–1259</code>) or from 'מחזורים פתוחים' in the feed	
4	'הצטרף' / 'הצטרף למחזור'	<code>:3033</code> / <code>:3415</code>

And step 4 may not end it either. If the club's game carries `requiresApproval`, tap 4 produces 'בקשת ההצטרפות נשלחה' (`:235`) and the user is `pending`, not registered — a **second** unbounded admin wait, during which the app tells them 'אתה רשום למחזור' (§5.1). If the club's game is `scheduled`, there is no button at all until the CF flips it (`MatchListCard.tsx:70`).

Summary: 2 taps in the ideal supply case, 6 self-service taps to a match with nobody in it, and in the realistic case 4 taps spread across up to two open-ended human approvals that the app gives the new user no way to track.

Open questions from this section

Things this reader could not settle from the code, listed rather than assumed.

- Does a brand-new account actually have `user.position` set at the end of profile setup? If it does, `statsCandidates` pushes a `position` candidate (src/utils/assistant/rules.ts:632-638), `statsRule` fires at priority STATS:5 and OUTRANKS the DISCOVERY:6 welcome rule — so the first sentence a new user ever reads would be a position line with a 'לסטטיסטיקה' CTA into an empty stats screen, and 'ברוך הבא ל-Teamder! 🙌' would never appear. This flips the entire opening impression and can only be settled by checking ProfileSetup on a device.
 - How many public, open, future games actually exist in production right now? `getOpenGames` (src/services/gameService.ts:2547-2554) is the ONLY query that can return anything for a user with no club, and a code comment at src/components/match/MatchListCard.tsx:73-76 says the one public game in the country is sitting at 21/21. If that is still true, the entire 2-tap best case is unreachable and every new user is forced down the club route.
 - What does the `availabilityCounts` callable return for an account with no saved home city? The client only branches on `hasLocation` (src/screens/tabs/ProfileScreen.tsx:803-807); the server function is not in this repo's client source. If it happens to return `hasLocation: true` with generic counts, the recommended-day banner and the podium DO render on the empty home and my §1.6/§1.8 mapping is wrong.
 - Current Remote Config values for `feature_quick_games`, `feature_friends`, `feature_referrals`, `feature_feedback`, and the five `games_feed_*` knobs. `feature_quick_games` in particular decides whether the create chooser offers any usable option at all to a user with no club (src/screens/games/GamesListScreen.tsx:1141).
 - What proportion of clubs in `/groupsPublic` have `isOpen: true`? The default in the create wizard is OFF (src/i18n/he.ts:1307-1308 + src/services/groupService.ts:2030), which is what makes the realistic join path pass through an unbounded admin wait. The actual split decides whether that wait is the common case or the exception.
 - Is a push notification sent to the requester when a club or game join request is approved or rejected? Nothing on the client renders an approval status, and the bell inbox is incoming-only (src/services/requestsService.ts:59-69), so push is the only possible channel — and it lives in the Cloud Functions source, not here.
 - Does `PostSignInOnboardingScreen` (src/screens/onboarding/PostSignInOnboardingScreen.tsx) already introduce clubs/games before the user reaches home? I did not map it — it belongs to the onboarding surface — but whether it primes the 'join a club' concept changes how the empty home reads.
 - Exactly what the three action tiles look like at real widths in Hebrew. The subtitles were removed because 'הצטרף למחזור' was clipping in a ~110pt column (src/components/home/HomeDashboardParts.tsx:145-148), and the titles still render with `numberOfLines={1}` (:166) — needs a device screenshot to confirm the longest label still fits.
 - Whether a guest can reach the games feed and tap 'הצטרף' on a card. `ensureNotGuest` is present in GamesListScreen's `handleCreate` (:315) and `openCreateForSlot` (:591) but NOT in `handleCardPrimary` (:337), so the card join appears ungated for a guest. Needs a device check against the Firestore rules to see what the write actually does.
-

5.5 · The web landing pages, from the source

0. Method, and what is verified against production

Everything below was read from the files in `/Users/matan/Projects/soccer` and checked against the live site with `curl` and headless Chrome at a true 390px CSS viewport.

Repo and production are in sync. `curl` of `/`, `/get` and `/app` returned bytes identical to `public/index.html`, `public/get.html`, `public/invite.html`. The two SSR templates are also in sync: `public/invite.html` is byte-identical to `functions/templates/invite.html`, and `public/c/index.html` to `functions/templates/community.html`, kept so by `functions/scripts/copy-template.js:20-29`, run from the build script at `functions/package.json:10` (`node scripts/copy-template.js && tsc`).

There is no custom domain. Every share link the app emits is built on `const HOSTING_ORIGIN = 'https://teamderfc.web.app' — src/services/deepLinkService.ts:44`, `src/services/inviteLinkService.ts:16`. The reason is on record at `src/services/deepLinkService.ts:30-35`: *"The bare 'teamder' subdomain was reserved by another Firebase project before we owned the brand, so we ship under 'teamderfc' (fc = football club)."* So every WhatsApp invite a real user sends shows a raw `.web.app` Firebase URL.

Screenshots at a true 390px viewport are in `/Users/matan/Projects/soccer/.landing-shots/`: `01-home-390.png` (`/`), `02-get-desktop-390.png` (`/get`, desktop rendering — mobile auto-redirects), `03-app-invite-generic-390.png` (`/app`), `04-privacy-390.png` (top 2400px of 11403px), `05-delete-account-390.png`, `06-community-showcase-390.png` (`/c/0Vn55bS62vvA3iBZsa3A`, a real live community), `07-community-notfound-390.png`.

A caution on Chrome: headless clamps its window to a 500px minimum, so naive `--window-size=390` screenshots are a 390px **crop of a 500px layout** and look falsely broken. All shots above were taken through a 390px iframe harness instead.

1. Routing — every public route and what answers it

From `firebase.json`. Two hosting targets, `teamderfc` and `soccer-app-52b6b`, carry **identical** rewrite blocks (`firebase.json:9-49` and `:50-90`), so both origins behave the same. `cleanUrls: true` and `trailingSlash: false` are set on both.

Route	Answered by	firebase.json
<code>/</code>	static <code>public/index.html</code>	(no rewrite; static)
<code>/get</code>	static <code>public/get.html</code>	(no rewrite; static)
<code>/app</code>	rewrite → static <code>/invite.html</code>	<code>:13</code>
<code>/go</code>	rewrite → static <code>/invite.html</code>	<code>:14</code>
<code>/session/**</code>	rewrite → static <code>/invite.html</code>	<code>:12</code>
<code>/i/**</code>	CF <code>serveInviteCode</code> (us-central1)	<code>:44-47</code>
<code>/c/**</code>	CF <code>serveCommunityPage</code>	<code>:40-43</code>
<code>/team/**</code>	CF <code>serveCommunityPage</code>	<code>:36-39</code>

Route	Answered by	firebase.json
/track-click	CF trackLinkClick	:15
/invite-preview	CF getInvitePreview	:16
/privacy	static public/privacy.html (via cleanUrls)	—
/delete-account	static public/delete-account.html	—

Verified live status codes: / 200, /get 200, /app 200, /go 200, /privacy 200, /delete-account 200, /c 200, /session/FAKEID 200, /team/FAKEID 200, /i/FAKEC0D 200, /c/FAKEID 200.

Two routing details worth flagging:

- /c (bare) is not the function. Firebase serves a matching static file before applying a rewrite, and public/c/index.html exists. So /c serves that file directly with **no SSR meta injection**, while /c/{groupId} falls through to serveCommunityPage. With no groupId the page's own script hits if (!groupId) { setState('error'); return; } (public/c/index.html:1006) and renders the "not found" state.
- /terms.html **does not exist**. find across the repo returns no terms*.html, and the live URL returns 404. The invite page footer links to it anyway (see §4.1).

2. / — the root landing page (public/index.html)

<html lang="he" dir="rtl"> at :2; viewport width=device-width, initial-scale=1.0 at :5. Font Heebo via Google Fonts :14.

The single most consequential fact about this page: it contains zero links to either app store. Grep counts across public/index.html: play.google.com → 0, apps.apple.com → 0. The only conversion path is a WhatsApp message asking to be added to a closed beta — while the app is in fact live on both stores (§6).

2.1 CONTENT, TOP TO BOTTOM, VERBATIM

Top nav (:400–411) — logo logo2.png :403; links פיצורים :406, איך זה עובד :407, הצטרפות לבטא :408. All three are in-page anchors (#features, #how, #beta). At ≤640px the entire link group is hidden by .nav-links { display: none; } (:390), so on a phone the top bar is a bare logo with no navigation and no CTA.

Hero (:414–443) — logo image :416; H1 across two lines: מארגנים משחק כדורגל :418 and, gradient-filled, קבוצה אחת, רישום למשחק בקליק, בחירת קבוצות חיה ודירוג חברים. הכול :419. Lead :422–423. Two CTAs: primary כבודקי בטא → href="#beta" :426,430; secondary יש באפליקציה → href="#features" :432–434. Below them a pill חינומי לחלוטין :440.

Note both hero CTAs are **in-page anchors**. Nothing on the first screen leaves the page.

Features (:446–524) — eyebrow פיצורים :448; H2 כל מה שצריך כדי לארגן משחק :449; sub סוף ל"מי :450. Six cards:

1. קהילות סגורות :463 — אדמין מאשר חברים. ובלי זרים. פותחים קבוצה לחברה הקבועים, מזמינים בקוד, ובלי זרים. אדמין מאשר חברים. חברים :464.

2. אתה רואה את המשחק הבא, נרשם, ויודע אם אתה ברשימה הראשית או בהמתנה. — 475: רישום למשחק בקליק. 476: בלי לעקוב אחרי וואטסאפ.
3. על המגרש: גוררים שחקנים בין קבוצות, מסמנים שוערים, ו-shuffle אוטומטי — 486: בחירת קבוצות חיה. 487: כשרוצים לערבב — מותאם 5v5/6v6/7v7.
4. כל חבר מדרג את האחרים בסקיל 1-5. הממוצעים אנונימיים, ועוזרים לאזן — 496: דירוגים בקהילה. 497: קבוצות אוטומטיות.
5. תזכורת שעה לפני, התראה כשמישהו מבטל ואתה ראשון בהמתנה, וכשפותחים משחק — 507: התראות חכמות. 508: חדש בקבוצה שלך.
6. כמה משחקים שיחקת, אחוז נוכחות, רצפים — וכרטיסי הישגים שמתפתחים עם — 518: הישגים וסטטיסטיקות. 519: השנים.

Card 4 is factually stale against the shipped product. It promises peer rating on a 1–5 scale; per `project_rating_1to10`, peer/crowd rating was deleted on 2026-06-24 and replaced by admin-only internal rating on a 1–10 scale.

How it works (:527–550) — eyebrow איך זה עובד :529 ; H2 מהורדה למשחק — שלוש דקות :530 . Steps: 1 פותחים :536 ; 2 שם, מגרש, תדירות. שולחים קוד הזמנה לחבריה — ובאמת זה הכול. / :535 פותחים קבוצה :541 ; 3 בוחרים תאריך, שעה, פורמט (5v5/6v6/7v7), והאפליקציה דואגת לרישום ולתזכורות. / :540 משחק על המגרש פותחים את מסך ה-Live, מחלקים קבוצות בכמה שניות, ומתחילים. / :545 מגיעים ומשחקים. 546: הסטטיסטיקות נשמרות אוטומטית.

Step 3 says the Live screen is where you split teams. Per `project_evening_flow_spec` the live match became timer-only in 2026-05 with teams/rotation removed, so this too is stale. The H2 also promises "from download to game", but the page never offers a download.

Beta section (:553–579) — badge רוצה להיות מהראשונים לנסות? H2 :556 ; בטא פתוחה • מקומות מוגבלים :557 ; body :559–560 : אוסיף. Google (של המכשיר). אתכם לרשימת הבודקים ותקבלו קישור להורדה.

Two CTAs. Primary :563 ; `https://wa.me/972546986121?text=...` → :567 שלחו את המייל בוואטסאפ שלחו את המייל שלי. Secondary :569 : המייל שלי Teamder&body= לבטא של Teamder.\n. או שליחה במייל. :574 → `mailto:studiogameslime@gmail.com?subject=`

Note the copy is first-person singular ("שלחו לי", "אויף אתכם") while the footer is corporate ("Studio Games Lime"). It asks for the device's **Google** account email — a Play closed-test mechanic with no iOS equivalent offered. A personal mobile number and a personal Gmail address are the only two contact channels.

Footer (:582–599) — brand :587 ; links → /privacy מדיניות פרטיות ; מארגנים משחקי כדורגל בלי בלגן • :587 ; → /delete-account מחיקת חשבון, :590 ; copyright © Studio Games Lime • :592 ; צור קשר → :591 ; כל הזכויות שמורות :596 .

2.2 CTA BEHAVIOUR BY PLATFORM

Identical on Android, iOS and desktop — no UA sniffing on this page. `#beta` and `#features` scroll in-page. `wa.me` opens WhatsApp (app on mobile, web on desktop). `mailto:` opens the mail client.

2.3 META / WHATSAPP PREVIEW

<title> :6 Teamder — מארגנים משחקי כדורגל בלי בלגן ; description :7 Teamder היא האפליקציה לארגון Teamder ; og:title :8 same as title; og:description :9 : משחקי כדורגל שכונתיים: קבוצות, רישום, בחירת קבוצות חיה ודירוג חברים — הכול במקום אחד קבוצות, רישום למשחק, בחירת קבוצות חיה ודירוגים. הכול במקום

.אָפּ; og:image :10 https://teamderfc.web.app/logo.png . No og:type , no og:locale , no twitter card. logo.png is 512×512 square, so the WhatsApp card renders as a small thumbnail, not a wide hero.

2.4 RTL / 390PX

Measured at a true 390px viewport: scrollWidth 390, body.scrollWidth 390, **0 overflowing elements**, document height 4315px. RTL is document-level via dir="rtl" with no per-element overrides. The step number badge uses the logical property inset-inline-start: 28px (:253), which is RTL-correct. Nothing breaks at 390px.

3. /get — the smart store link (public/get.html)

Its own header comment states the intent (:2-7): *"clean, marketing-friendly smart store link. Detects the platform and redirects straight to the right store... No invite framing, no app-open attempt — purpose-built for 'try my app' posts (Facebook groups etc.)"*.

Nothing in src/ references /get ; grep for web.app/get across the repo returns nothing. It is a hand-pasted marketing URL only.

3.1 CONTENT

<html lang="he" dir="rtl"> :8 . Title Teamder · הורדה :12 . Logo :40 ; H1 Teamder :41 ; body :42
מארגנים כדורגל בלי בלגן then, on a second line, פותח לך את החנות... Two buttons: הורדה ל- iPhone 🍏
:44 and הורדה ל- Android 🤖 :45 . Hint :47 . לא נפתח אוטומטית? בחר/י את החנות שלך למעלה .

The Hebrew mixes hyphen forms: iPhone-ל / Android-ל here use an ASCII hyphen, while the rest of the estate uses the maqaf - / - (e.g. invite.html:159).

3.2 STORE URLS AND PLATFORM BEHAVIOUR

Both defined twice — as href and again as JS constants: iOS https://apps.apple.com/app/id6775178022 (:44 , :51), Android https://play.google.com/store/apps/details?id=com.studiogameslime.soccerapp (:45 , :52).

UA sniffing at :53-65 : isAndroid = /Android/i.test(ua) ; isIOS = /iPhone|iPad|iPod/i.test(ua) || (/Macintosh/.test(ua) && navigator.maxTouchPoints > 1) — the second clause deliberately catches iPadOS 13+, which reports as Mac (:56 comment).

- **Android:** hides the iOS button, then location.replace(ANDROID) :60-61 . Zero taps.
- **iOS / touch Mac:** hides the Android button, then location.replace(IOS) :63-64 . Zero taps.
- **Desktop:** no redirect, both buttons shown (:66 comment). This is what 02-get-desktop-390.png shows.

No install referrer and no invitedBy is attached on this path, so a /get install is attributed to nothing.

3.3 META

Title :12 ; description :13 Teamder — מארגנים כדורגל בלי בלגן . הורד עכשיו . ; og:title :14 Teamder · מארגנים משחק, מחלקים כוחות ומוצאים שחקנים — באפליקציה אחת . ; og:description :15 כדורגל בלי בלגן ; og:image :16 /logo.png .

3.4 RTL / 390PX

Measured: `scrollWidth` 390, 0 overflowing elements. `body` is a flex centering box with `min-height:100vh` (:21-23), card `max-width:380px` (:25) — fits 390px with 24px body padding. Buttons are a full-width column (:29).

4. The invite landing page (`public/invite.html`) — four routes, five copy variants

One 25KB file backs `/app`, `/go`, `/session/**` (all static rewrites), `/i/**` (via `serveInviteCode`) and `/team/**` (via `serveCommunityPage`). It is the page that actually receives invited users, so it carries the most behaviour.

4.1 STATIC CONTENT

`<html lang="he" dir="rtl">` :2; viewport includes `viewport-fit=cover` :5; `theme-color #0B1B3B` :16; font Assistant :19.

Header (:152-155) — logo pair; button הורד :154.

Hero (:157-168) — badge Teamder לברוכים הבאים :159; H1 כדורגל קבוע. בלי כאב הראש של הארגון. :160; lead פותחים משחק, החברה נרשמים ו-Teamder מטפלת בכל השאר. :162; an empty context card :164; a hidden note הקישור :164; iPhone ו-Android חינם :163; trust line הורד את Teamder :165; phone mock with `/shot-match.jpg` :167.

Problem → solution (:171-180) — H2 מ-40 הודעות למשחק אחד מסודר :172; three chat bubbles מי מגיע? :173; badge המשחק מלא 10/10, שחקנים :175; then the payoff card :178-179; כדור? :175; מוכנות • אתם רק מגיעים לשחק.

Final CTA (:183-190) — H2 פחות הודעות.
כדורגל. :184; over the store name: Google Play :187, App Store :188. :185; two store cards, each במקום אחד.

Footer (:191) — © Teamder • then פרטיות → `/privacy.html` and תנאי שימוש → `/terms.html`. That file does not exist in the repo and the live URL returns 404. Every invited user sees a dead "Terms of Service" link.

Sticky bar (:193) — appears once the hero has scrolled off and the final CTA is not yet near, per :262.

Note the CSS carries a whole unused chapter: `.action`, `.step`, `.stepN`, `.step.shot` (:104-115) are styled, and :255 observes `.step` elements — but no `.step` markup exists in the body. A three-step section was removed and its CSS left behind. So the page is only hero → problem → store, with no explanation of what the app does between them.

4.2 THE FIVE VARIANTS

Defined at :233-237, applied by `applyV()` :238 and called unconditionally at :239.

Variant	badge	title	CTA	final	shot
personal_invite :233	הזמנה אישית	הוזמנת ל-Teamder	הצטרף ל-Teamder	הצטרף לחברים שלך ב-Teamder	<code>/shot-home.jpg</code>
game :234	הזמנה למשחק	הוזמנת למשחק כדורגל	פתח את המשחק	המשחק הבא שלך כבר מחכה	<code>/shot-match.jpg</code>

Variant	badge	title	CTA	final	shot
community :235	הזמנה למועדון	הוזמנת להצטרף למועדון	פתח את המועדון	הצטרף למועדון דרך Teamder	/shot-games.jpg
generic :236	ברוכים הבאים ל-Teamder	כדורגל קבוע. בלי כאב הראש של הארגון.	הורד את Teamder	פחות הודעות. יותר כדורגל.	/shot-match.jpg
campaign :237	identical to generic in all five fields				/shot-match.jpg

Subtitles: **personal** משחק שכונתי מחכה לך. **game**; האפליקציה שמארגנת את הכדורגל כדי שאתם רק תגיעו לשחק. **community**; הצטרף לקבוצה קבועה וקבל עדכונים על משחקים חדשים. **generic/campaign** פותחים משחק, החבר'ה נרשמים ו-Teamder מטפלת בכל השאר.

With an inviter name resolved, :238 overrides badge to {name} הזמין אותך and title to {name} הזמין אותך ל-Teamder.

The five variants resolve to only two distinct images. md5 over public/: shot-home.jpg and shot-games.jpg are byte-identical (131533864dd6074e6ad9c5f0f6f94552), and shot-match.jpg and shot-teams.jpg are byte-identical (0e3bb2395149a1efe27feddd87bba418). So personal and community show the same picture, and game/generic/campaign show the same picture. All four are 540×1134. shot-teams.jpg is referenced by nothing (grep over public/ finds no reference) yet is served live (200).

Variant selection is at :214: session → game, team → community, else invitedBy → personal_invite, else source|campaign → campaign, else generic.

4.3 PER-ROUTE BEHAVIOUR

Type resolution :200–213: window.__INVITE__ wins if the server injected it, otherwise the first path segment.

- /app — static. type='app', isApp=true, valid=true, pathPart='app'. No code, so no preview fetch. Renders generic (verified — 03-app-invite-generic-390.png).
- /app?invitedBy=<uid> — ctxType='personal_invite' via :214. Title is the generic Teamder ל-הוזמנת; the **inviter's name is never resolved** on this path, because the name only arrives from /invite-preview, which :244 gates on code.
- /go — pathPart='go', deep link footy://go. Same generic copy as /app.
- /session/{id} — static, so **no OG injection**; ctxType='game' from the path, so the game copy applies, but the detail card (date / pitch / free spots) never renders, again because there is no code.
- /team/{id} — SSR'd by serveCommunityPage, which injects OG but **not** __INVITE__; ctxType='community' from the path. Community name, city and member count never render, same cause.
- /i/{code} — the only route that gets the full experience: serveInviteCode injects __INVITE__ and per-link OG, and code is set so the preview fetch runs and fills the detail card.

Since src/services/inviteLinkService.ts:58 makes /i/{code} the form of every share the app emits, the rich path is the common one — and /session/, /team/, /app are the degraded fallbacks used when the short-link write fails (:62–65).

4.4 THE /INVITE-PREVIEW FETCH AND THE CONTEXT CARD

:244–250. Only when code is truthy. On response:

- `type==='game'` and not past → `retarget('session', d.id)`, `game` copy, and a chip row built from `startsAt` (weekday + time via `Intl.DateTimeFormat('he-IL') :240-241`), `fieldName` or `city`, `communityName`, and `{n}` מקומות פנויים styled green when `availableSpots > 0` :246.
- `type==='game'` and `past` (`finished / cancelled / startsAt < now`) → if the game has a `communityId`, it **silently re-points to the parent community** (`retarget('team', ...)`, `community` copy, chips for city and `{n}` חברים); otherwise `toGeneric()` :246. A neat save: a stale game link becomes a community invite rather than a dead end.
- `type==='community'` → `community` copy plus city and `{n}` חברים chips :247.
- `type==='personal'` → `personal_invite` with the inviter's name :248.
- anything else, or a fetch rejection → `toGeneric(wasGameLink|wasCommLink)` :249-250, which reverts the copy, hides the card, and shows `Teamder` הקישור כבר לא זמין, אבל אפשר למצוא משחקים נוספים ב-Teamder :243, 165.

4.5 THE PRIMARY CTA — `PRIMARY()` AT :231

Wired to all three buttons (`ctaBtn`, `stickyBtn`, `hdrBtn`) at :232.

iOS. Writes the canonical https invite to the clipboard via `navigator.clipboard.writeText(clipUrl())`, then goes **straight to the App Store** — it never attempts `footy://`. The inline comment at :231 explains why: "iOS Safari throws a visible 'cannot open page' error for an unhandled custom scheme (footy://) when the app isn't installed — and an invite recipient never has it yet." Consequence: **an iOS user who already has Teamder installed is still sent to the App Store**, not into the app.

Android and desktop. :231: fire `track('landing_app_open_attempt')`, arm a 1500ms timer to `location.href = storeHref`, register a `visibilitychange` listener that cancels the timer if the page hides (the app opened), then `location.href = 'footy://...'`. On desktop this attempts an unknown protocol — browser-dependent, typically an error prompt — and then lands on the Google Play **web** listing 1.5s later.

If `valid` is false, it skips the scheme and goes straight to the store.

4.6 ATTRIBUTION MACHINERY

Play install referrer :229. With a campaign `source`, the referrer is `utm_source=...&utm_campaign=...&g=<gameId>&l=<linkId>`; otherwise `invite_app` or `invite_{type}_{id}`, with `_by_{invitedBy}` appended. `retarget()` :223-226 rewrites it once the preview resolves the real entity. Consumed on first launch by `src/services/installReferrerService.ts`.

iOS clipboard deferred deep link. The web side writes `https://teamderfc.web.app/{pathPart}{appQuery}` (`clipUrl()` :219). The app side is `src/services/clipboardInviteService.ts`: iOS only (:39), behind the Remote Config kill-switch `feature_ios_clipboard_invite` (:40), latched to one read per install (:42), skipped if a pending invite already exists (:46-50), and gated behind `Clipboard.hasUrlAsync()` (:56) — which, per the comment at :52-55, checks for a URL **without** triggering iOS 16's "Allow Paste?" prompt. On a match it stashes the invite and blanks the clipboard (:67-72); a non-matching URL is left alone (:70).

`localStorage.pendingInvite` :227 — written by the web page. Web `localStorage` is not readable by the native app, so this cannot serve attribution; it is only useful to the page itself on a return visit.

Click beacon :209. If any of `source`, `linkId` or `invitedBy` is present, fires `navigator.sendBeacon('/track-click?l=...&s=...&inviter=...')`, falling back to `fetch(..., {mode:'no-cors', keepalive:true})`.

4.7 ANALYTICS

`track()` :198 pushes to `window.dataLayer` and calls `gtag` if present. Events: `landing_view` :216, `landing_primary_cta_click` :231, `landing_app_open_attempt` :231, `landing_store_click` :231, `landing_context_loaded` :246–248, `landing_context_failed` :243, `landing_scroll_depth` at 25/50/75/100% :260.

`window.__AB__` :215 carries `context_type`, `campaign_source`, `platform` (`android / ios / web`), `has_inviter`, `has_game`, `has_community`.

A campaign source may arrive base64url-encoded as `?b=` and is decoded at :206–207, falling back to `?s=` then `?utm_source=`.

4.8 META / WHATSAPP PREVIEW

Static defaults: `title` :6 `Teamder · מארגנים כדורגל בלי כאב ראש`; `description` :7 `פותרים משחק, החברה`; `og:title` :9; `og:description` :10 `נרשמים ו-Teamder מטפלת בכל השאר - הרשמות, רשימת המתנה, חלוקת כוחות וסטטיסטיקות.`; `og:image` :11 `/logo.png`; `og:locale he_IL` :12; `twitter:card summary_large_image` :13.

Verified live, `/session/FAKEID` returns exactly these static tags — **no per-game injection**, because that route is a static rewrite.

4.9 RTL / 390PX

Measured: `scrollWidth` 390, 0 overflowing elements, document height 2021px. One content width `--cw:342px` with `max-width: calc(100% - 44px)` (:25,47) — 342px fits inside 390px. Headings are explicitly right-aligned at :48. Sticky bar respects `env(safe-area-inset-bottom)` :128. A `prefers-reduced-motion` block at :139–143 forces all revealed content to `opacity:1`.

I checked the reveal animations specifically, since the bubbles and payoff card start at `opacity:0` and depend on an `IntersectionObserver` (:253, threshold 0.28). Measured over time in a 390px viewport: bubbles reach opacity 1, `.down` 1, `.order` 1 by $t \approx 1600\text{ms}$. **They work** — an early screenshot that appeared to show them missing was a capture-timing artifact, not a defect.

5. `/i/**` — `serveInviteCode` (`functions/src/index.ts:10687–10825`)

Resolves a 7-char base62 code (`src/services/inviteLinkService.ts:17–28`) against `inviteLinks/{code}` :10699. A pure alias, no redirect, so the short URL stays in the address bar (:10680–10686).

Side effects on every hit (:10711–10740): increments `clicks` and `lastClickAt` on the link doc; increments `inviteClicks/{invitedBy}` when an inviter is known — the comment at :10718–10724 records that short links used to skip this, so a person's profile counter read 7 while the aggregate read 15; and bumps the cross-source daily aggregate via `bumpLinkClickAggregate`.

SSR OG by link type (:10748–10799): - `team` → `loadShowcaseSummary` → `{name} · Teamder` plus the group description, or `{n} חברי סגל · Teamder · {city} · {n}` (:10542–10552), with the group cover as `og:image`. - `session` → reads `games/{id}`; `title {title}` `הוזמנת למשחק` or `הוזמנת למשחק` `Teamder · {where}` `משחק כדורגל ב-`; `description` `Teamder · ראה מי מגיע והצטרף בלחיצה.` `כדורגל ב-`; `og:image` — game links fall back to the 512×512 logo. - `inviter only` → reads the user's name; `{name}` `Teamder` `הזמנה אישית` or `מזמין אותך ל-`.

Teamder-ל; description האפליקציה שמארגנת את הכדורגל כדי שרק תגיעו Teamder – ב-Teamder שלד לחברים שלך ב-Teamder / כדורגל קבוע בלי כאב ראש • Teamder → -else (:10788–10792). לשחק. פותחים משחק, החבריה נרשמים / Teamder מטפלת בכל השאר. (:10794–10798)

Note the OG copy says הזמין אותך (:10789) while the page body says הזמין אותך (invite.html:238) — present vs past tense for the same person.

Injects <script>window.__INVITE__={...}</script> before </head> :10804–10809 . Cache-Control: public, max-age=60, s-maxage=60 :10813 . On error, serves the bare template :10817–10819 — which silently drops attribution; src/services/inviteLinkService.ts:52–57 compensates by also putting ?invitedBy= in the URL.

Verified live: /i/FAKEC0D returns title Teamder • כדורגל קבוע בלי כאב ראש and __INVITE__= {"type":"app","id":"","invitedBy":""} . An unknown code degrades silently to the generic page with a 200 — no "this link expired" state.

6. /c/** and /team/** — serveCommunityPage (functions/src/index.ts:10619–10678)

One function, two templates, chosen by isInvite = parts[0] === 'team' :10630 : /team/{id} → invite.html, /c/{id} → community.html :10636–10638 . Cache-Control: public, max-age=300, s-maxage=600 :10657–10660 .

The fallback meta is wrong for /team/. buildMetaBlock :10531–10554 returns, when no showcase doc exists, title Teamder מועדון and description והמשחקים, השחקנים הכי נאמנים, צפו בסטיסטיקות המועדון, השחקנים הכי נאמנים, והמשחקים That is browse-the-stats copy, correct for /c/ and wrong for /team/, which is a join invite. Verified live: /team/FAKEID returns exactly that description.

6.1 THE SHOWCASE PAGE (PUBLIC/C/INDEX.HTML)

No in-app entry point. Grep across the whole repo for web.app/c/ finds one match, a comment at functions/src/index.ts:9977 . Nothing in src/ ever shares a /c/{id} URL. The page is reachable only by hand-built link.

Vocabulary conflict. The page says קהילה throughout — <title> :7 Teamder-ב קהילה, og:title :12, error הקהילה לא נמצאה :791, empty הקהילה עדיין נבנית :804, CTA הצטרפו לקהילה :843, section הקהילה :843, body :931. The SSR function that renders it says מועדון (:10537, :10551). Verified live at /c/FAKEID : SSR title Teamder-ב מועדון, page body הקהילה לא נמצאה . The app's own strings also say מועדון (src/i18n/he.ts:1568, :1770).

Content. Nav logo + פתח באפליקציה :772 . Loading ... טוען את הקהילה :782 . Error הקהילה לא נמצאה :791 / Empty :793 . לדף הבית / :792 ייתכן שהקישור שגוי או שהקהילה הוסרה. נסו לפתוח את הקישור מחדש. :805 אחרי המשחק הראשון תהיה כאן עמוד יפה עם סטיסטיקות. חזרו בקרוב. / :804 הקהילה עדיין נבנית :806 . פתח באפליקציה

Hero: eyebrow קהילת כדורגל פעילה :834 ; H1 from data :835 ; chips for city, pitch, {year} מאז :1157 ; CTAs :846 . הורדת האפליקציה and :843 הצטרפו לקהילה

Stats :882 . משחקים החודש, :875 אחוז הצלחה, :868 חברים פעילים, :861 משחקים שנערכו :853–886 . is never defined anywhere on the page; it maps to organizationRatePct :1171 .

Sections: :891–893 ; השחקנים שהכי הופיעו למשחקים בשנה האחרונה. / אלופי הנוכחות / הכי נאמנים למגרש :891–893 ; מי / הקהילה :902–904 ; כל משחק עם תאריך ומספר שחקנים שהגיעו. / מה קרה לאחרונה / משחקים אחרונים פתחו את הקישור / רוצים להצטרף? :919–921 ; final חברי הקהילה – מנהלים מסומנים בזהב. / משחק כאן

Teamder ותצטרפו ישר לקהילה — בלי קבוצות וואטסאפ אינסופיות. פתח :930–931 with

באפליקציה :945. הורד את Teamder and :937 באפליקציה

Footer :955–972 : עמוד ראשי :963, מדיניות פרטיות :964, הורד את האפליקציה → hardcoded Google Play :965.

A live ad placeholder. :911–913 renders `<div class="ad-slot"><span>שומר למודעה</span></div>` — a dashed grey box reading "space reserved for an advert", styled at :588–601, visible on real community pages.

Confirmed in 06–community–showcase–390.png. public/app-ads.txt carries a real AdSense publisher id.

Store links. The markup has five play.google.com links and one apps.apple.com — the latter only as the JS constant APP_STORE_URL :985. Two JS passes patch this: :1038–1041 rewrites every

a[href*="play.google.com"] to the App Store on iOS, and :1075–1081 re-resolves heroDownload and finalDownload per platform (the comment at :1071–1074 notes the first pass would otherwise be undone). The footer link :965 is covered by the first pass. So iOS is handled, but only by JS — with JS disabled every visitor gets Google Play.

Auto deep-link on every mobile visit :1042–1064. On Android Chrome it fires

location.replace('intent://team/{id}#Intent;scheme=footy;package=...;S.browser_fallback_url=<this page>;end'). On other mobile browsers it assigns footy://team/{id} — which, on browsers that surface unhandled schemes, shows an error for a visitor without the app. Unlike invite.html there is no store fallback, by design (:1025–1032).

Data. Fetched client-side from the Firestore REST API with a public API key :982–983, :1084–1087. A 404 sets the error state, a doc with no name sets empty :1096–1098.

6.2 TWO DEFECTS CONFIRMED LIVE AT 390PX

Both verified on a real community, /c/0Vn55bS62vvA3iBZsa3A ("כדורגל קריית עקרון"), in 06–community–showcase–390.png.

- The podium puts 2nd place above 1st on mobile.** renderPodium builds DOM order [silver, gold, bronze] (:1217–1220) precisely so a 3-column grid centres gold (:1214–1216, grid at :347). At ≤720px :352–353 collapses it to grid-template-columns: 1fr, and the DOM order becomes vertical order — so the reader meets the #2 card, then #1, then #3. The screenshot shows the silver "2" card (מתן לוי) stacked above the gold "1" card.
- Hebrew singular is never handled.** Rendered live: משחקים 1 ("1 games"). Sources: :1228 '' + (entry.p.gamesPlayed || 0) + ' משחקים · ', :1242, :1264 (g.attendedCount || 0) + ' משחקנים', :1301 games + ' משחקים'. The app itself does this correctly — src/i18n/he.ts:1574–1575 communityMembersCount: (n) => n === 1 ? 'שחקן אחד' : \ \${n} שחקנים — so the web page is out of step with the product's own standard.

6.3 /C/ RESPONSIVE

Breakpoints at :269 (stats 4→2 cols), :352 (podium→1 col), :751–758 (padding). Members grid repeat(auto-fill, minmax(140px,1fr)) :530; recent-games titles ellipsis at :505–506.

7. /invite-preview — getInvitePreview (functions/src/index.ts:13124–13254)

CORS-enabled :13125 , Cache-Control: public, max-age=60 :13127 . Returns {type:'generic'} for a missing or absent code :13131–13139 , and — notably — for **any** internal error too :13249–13251 (*"Never leak an internal error — degrade to the generic page"*).

- session :13150–13207 : computes occupancy as players + non-cancelled non-waitlisted guests + pendingPromotion :13157–13165 , returns availableSpots = max(0, maxPlayers - occ) , plus title, startsAt , fieldName , city , status , and the parent community from groupsPublic so a past game can fall back to it (:13166–13169).
- team :13209–13229 : name, city, cover, memberCount .
- personal :13234–13245 : returns **only** the inviter's display name — the comment at :13231–13233 notes no uid or phone is exposed.

8. /track-click — trackLinkClick (functions/src/index.ts:13256–13301)

Reads ?l= (ad link), ?s= (legacy source), ?inviter= (uid). Writes adLinks/{l}.clicks or linkClicks/{s}.clicks :13269–13278 , always bumps inviteClicks/{inviter} when present :13280–13284 , and bumps the daily aggregate :13289–13294 . Always returns 204 with Cache-Control: no-store :13298–13299 ; every failure is swallowed (*"best-effort beacon — never error the user's redirect"* :13296).

9. Cross-cutting

9.1 DEEP-LINK REGISTRATION — SHORT LINKS NEVER OPEN THE INSTALLED APP

app.json registers schemes footy , teamder , com.studiogameslime.soccerapp . Verified Android intent filters cover exactly three path prefixes on teamderfc.web.app : /session , /team , /app (autoVerify: true). iOS associatedDomains is applinks:teamderfc.web.app , and public/.well-known/apple-app-site-association lists paths ["/session/*", "/team/*", "/app"] .

Neither platform registers /i/* or /go . But /i/{code} is the form of *every* share the app emits (src/services/inviteLinkService.ts:58 , called from ProfileScreen.tsx:929 , FriendsScreen.tsx:314 , MatchDetailsScreen.tsx:1472 and :1655 , CommunityDetailsScreen.tsx:456 , onboardingService.ts:53). So a recipient who **already has Teamder installed** and taps a shared link always lands in the browser on invite.html first, and must tap the CTA to be handed off.

Compounding it, parseInviteUrl (src/services/deepLinkService.ts:103–194) handles /session/* , /team/* , /app , /go — but has no /i/ branch, so it returns null for a short link. This is currently harmless because the clipboard path writes the **long** form (invite.html:219), but it means a pasted short link cannot be recovered.

assetlinks.json declares two SHA-256 fingerprints (upload + Play signing).

9.2 JORYIO WEB TRACKING (PUBLIC/JS/JORYIO.JS)

Loaded with defer on index :601 , get :69 , invite :267 , delete-account :323 . **Not** on privacy.html and **not** on public/c/index.html — so the community showcase, the one page with real content worth measuring, is untracked.

It posts to `https://hippomation-backend.fly.dev/api/v1/track` :19 with a hardcoded bearer key :20 . Anonymous id in `localStorage` , regenerated per-session in private mode :30–34 . It wraps `dataLayer.push` and replays earlier pushes :118–127 , converting `snake_case` to Title Case :110–114 . It emits its own `Landing View` only when `__INVITE__` is absent or empty — the comment at :129–133 flags the trap that "*the pages define it as {} when there is no invite, and an empty object is truthy.*" A capture-phase click listener :138–147 classifies outbound links into `Store Link Clicked` (with `store: android|ios`) , `App Deep Link Clicked` , `Whatsapp Link Clicked` .

9.3 LEGAL PAGES

`/privacy` (`public/privacy.html`) — title :6 `Teamder – מדיניות פרטיות` ; eyebrow משפטי :233 ; H1 :235 . Eleven sections :244–403 :249 , פרטים שנאספים , (with פרטים שאתם מספקים ישירות) מה אנחנו אוספים , מי אנחנו :263 , אוטומטית נתוני בריאות וכושר , :288 , דפי קהילה ציבוריים , :278 , שער `Wear OS` ווידג'ט הטלפון , :263 , כמה זמן , היכן המידע נשמר , עם מי המידע משותף , למה משמש המידע , (`Health Connect / HealthKit`) :291 , `11,403px tall at 390px` — יצירת קשר , שינויים במדיניות , אבטחה , ילדים , הזכויות שלכם , נשמר המידע roughly 13.5 phone screens. No OG tags, no analytics. The nav still points at `/#features` , `/#how` , `/#beta` (:223–225).

Two staleness notes: the date is 29 May 2026, predating the Wear OS removal, the rating change, and the Joryio SDK; and §2.4 is headed `דפי קהילה ציבוריים` while the SSR calls the same thing מועדון.

`/delete-account` (`public/delete-account.html`) — title :6 ; eyebrow חשבון :258 ; H1 מחיקת חשבון :260 . אפשר למחוק את החשבון ואת הנתונים שלכם בכל רגע , ישירות מתוך האפליקציה . :259 ; lead

Steps :271–274 : גללו לתחתית המסך / עברו לטאב "הפרופיל שלי" . / פתחו את `Teamder` והיכנסו לחשבון . :271–274 : אשרו את הפעולה . החשבון יימחק תוך מספר שניות . / והקישו על "מחיקת חשבון" .

Table / פרטי הפרופיל (שם , אימיל , טלפון , עיר , רמת מיומנות) — נמחק מיד :279–297 : מה נמחק ומה נשמר / דוחות קריסה — נשמר ; תוך שניות / חברויות בקבוצות והיסטוריית הרשמות — נמחק מיד ; תוך שניות משחקים שיצרת/ניהלת — נשמר ; עד 14 חודשים / אנונימיים (`Crashlytics`) ותעבורה אנונימית (`Analytics`) . קבוע / נשארים בקבוצה אך ללא שמך (מוחלף ב-"משתמש שהוסר") .

This page describes deletion as in-app only and offers no web form, which is a gap against Google Play's data-deletion URL requirement (the URL must let a user *request* deletion without the app).

9.4 EXPOSED AND ORPHANED ASSETS

- A pre-release Android build is publicly downloadable. `public/downloads/teamder-0.2.5.aab` , 44,061,763 bytes, served live (range request returned 206). Anyone who guesses or finds the path gets a v0.2.5 build of the app.
- `public/app-preview.png` (1080×2400) and `public/shot-teams.jpg` are referenced by nothing in `public/` yet both return 200.
- `logo.png` is 512×512; `logo2.png` is 1969×799. Every `og:image` on the estate points at the square `logo.png` , so no surface produces a wide WhatsApp hero card.

9.5 STORE URLS — BOTH REAL

Android `https://play.google.com/store/apps/details?id=com.studiogameslime.soccerapp` — live, 200.

iOS `https://apps.apple.com/app/id6775178022` — **real, not a placeholder**. The iTunes lookup API returns `resultCount: 1, trackName: Teamder, bundleId: com.studiogameslime.soccerapp, version: 1.1.7, released 2026-09-16` . (A direct `curl` returns 429 rate-limiting, not a 404.) The same id appears at `get.html:44,51, invite.html:188,231, c/index.html:985` .

Which makes the root landing page's closed-beta framing (§2.1) the estate's sharpest contradiction: the app has been publicly downloadable on both stores for months, and `teamderfc.web.app` still asks visitors to WhatsApp a phone number for a Play test-track invite.

10. Summary table

Route	Rendered by	Primary CTA (Hebrew)	Where that CTA sends an Android user
<code>/</code>	static <code>public/index.html</code>	(:430) הצטרפו כבודקי בטא → anchor <code>#beta</code> , then שלחו את (:567) המייל בוואטסאפ	WhatsApp chat to +972546986121 — never a store
<code>/get</code>	static <code>public/get.html</code>	none needed	Auto <code>location.replace</code> → Google Play (0 taps, :60-61)
<code>/app</code>	rewrite → static <code>invite.html</code>	Teamder הורד את (:163)	<code>footy://app</code> , then Google Play after 1500ms (:231) , referrer <code>invite_app</code>
<code>/go</code>	rewrite → static <code>invite.html</code>	Teamder הורד את	<code>footy://go</code> , then Google Play after 1500ms
<code>/session/{id}</code>	rewrite → static <code>invite.html</code> (no SSR)	(:234) פתח את המשחק	<code>footy://session/{id}</code> , then Google Play, referrer <code>invite_session_{id}[_by_{uid}]</code>
<code>/i/{code}</code>	CF <code>serveInviteCode</code>	variant-dependent: פתח את / פתח את המועדון / המשחק Teamder-ל הצטרף	<code>footy://{type}/{id}</code> , then Google Play with the retargeted referrer (:223-226)
<code>/team/{id}</code>	CF <code>serveCommunityPage</code> → invite template	(:235) פתח את המועדון	<code>footy://team/{id}</code> , then Google Play, referrer <code>invite_team_{id}</code>
<code>/c/{id}</code>	CF <code>serveCommunityPage</code> → community template	(:843) / הצטרפו לקהילה (:846) הורדת האפליקציה	Auto <code>intent://team/{id}</code> on Chrome (:1046-1057) ; download button → Google Play + <code>referrer=invite_team_{id}</code> (:1069-1070)
<code>/c</code> (bare)	static <code>public/c/index.html</code>	—	Error state הקהילה לא נמצאה
<code>/privacy</code>	static	—	—
<code>/delete-account</code>	static	—	—

Route	Rendered by	Primary CTA (Hebrew)	Where that CTA sends an Android user
/invite – preview	CF getInvitePreview	—	JSON
/track-click	CF trackLinkClick	—	204
/terms.html	nothing — 404	—	Linked from invite.html:191

Open questions from this section

Things this reader could not settle from the code, listed rather than assumed.

- Is the root landing page's closed-beta framing intentional? The app has been live on both stores for months (iOS 1.1.7 released 2026-09-16, Android live on Play production), yet teamderfc.web.app/ carries zero store links and routes every visitor to a WhatsApp message asking for a Google-account email. I could not determine from the code whether this is a forgotten page or a deliberate funnel; it needs an owner decision.
- What share of real traffic reaches each route? The code tells me /i/{code} is the form of every in-app share and that /get is hand-pasted into Facebook groups, but the actual split between /, /get, /i/, /session/, /team/ and /c/ can only come from the Joryio dashboard or Firebase Hosting logs.
- Does the iOS clipboard deferred deep link actually work end to end on a real device? The web write is inside a click handler, but navigator.clipboard.writeText is async and location.href fires immediately after (invite.html:231). Only a device test can confirm the write lands before navigation. Also unverified: whether the Remote Config flag feature_ios_clipboard_invite is currently ON in production.
- Does the Android intent:// auto-redirect on /c/{id} (c/index.html:1046-1057) behave well when the app is NOT installed? The S.browser_fallback_url points back at the same page, which should be a no-op, but I could not test this on a real Android device.
- On non-Chrome Android browsers and on desktop, does the raw footy:// scheme assignment produce a visible error dialog before the 1500ms store fallback fires (invite.html:231, c/index.html:1062)? The iOS path was explicitly changed to avoid exactly this, so the same risk on other platforms needs a device check.
- Should the public /downloads/teamder-0.2.5.aab (44MB, served live, returns 206) be removed? I verified it is downloadable but cannot tell whether it is still deliberately linked from somewhere outside this repo.
- Is 'אחוז הצלחה' (c/index.html:875, backed by organizationRatePct) a metric a visitor can interpret? It is never defined on the page and I could not find a user-facing definition anywhere.
- Which vocabulary should win, מועדון or קהילה? The app strings and the SSR function say מועדון; the entire /c/ showcase page and the privacy policy section 2.4 say קהילה. Both render on the same URL today. This needs an owner ruling before copy is fixed.
- Are the Open Graph previews actually rendering as intended in WhatsApp? I verified the served meta tags per route, but WhatsApp's crawler caches aggressively and no route supplies a wide image (every og:image is the 512x512 logo.png, and /i/ game links supply no og:image at all) — worth a real paste test.
- Does the /team/{id} route ever get shared in practice, or has the short link fully replaced it? It matters because its SSR fallback meta is community-showcase copy on a join-invite page (functions/src/index.ts:10535-10540).